TrpB MetaDynamics 复刻教程

项目：复刻 *Pyrococcus furiosus* TrpB 的 COMM 域构象景观（conformational landscape）
参考文献：Maria-Solano, Iglesias-Fernandez & Osuna, *JACS* 2019, 141, 13049–13056
DOI: 10.1021/jacs.9b03646
Supporting Information: ja9b03646_si_001.pdf
作者：Zhenpeng Liu (liualex@ad.unc.edu), UNC Chapel Hill
最后更新：2026-04-02

> **重要提示**：本教程中所有路径使用 `$WORK` 作为你在 Longleaf 上的工作目录简写。开始之前，请先设置这个变量：
>
> ```
> export WORK=/work/users/<首字母>/<前两个字母>/<你的 ONYEN>/AnimaLab
> # 例如: export WORK=/work/users/j/jo/johndoe/AnimaLab
> ```
>
> 将 < 你的 ONYEN> 替换为你的 UNC ONYEN。目录名称可以随意取——你可以用任何项目名称代替“AnimaLab”。

如何阅读本文档

- 每个命令都**可直接复制粘贴**，包含完整绝对路径。
- 每个输入文件都以 `cat > filename << 'EOF' ... EOF` 的方式内联写出。
- 每个参数都标注了**来源标签**：`[SI p.S2]`、`[SI p.S3]`、`[Caulkins 2014]` 或 `[operational default]`。
- Phase 0–5 已在 UNC Longleaf 上**验证通过**（截至 2026-04-01）。
- Phase 6–9 标注为 **[DRAFT –not yet verified]**（草稿，尚未验证）。
- 每一步结束时都有**验证命令**（`grep`、`ls -lh`、`wc -l`）。

目录

1. 前置条件
2. Phase 0: 环境搭建（~30 分钟）
3. Phase 1: 下载与准备 PDB 结构（~10 分钟）
4. Phase 2: PLP 辅因子参数化（~2–3 天）
5. Phase 3: 用 tleap 构建体系（~5 分钟）
6. Phase 4: 经典 MD –前处理流程（~24 小时）
7. Phase 5: 经典 MD –500 ns 产出运行（~3 天）
8. Phase 6: AMBER 到 GROMACS 格式转换（DRAFT）（~30 分钟）
9. Phase 7: Path CV 构建（DRAFT）（~10 分钟）
10. Phase 8: Well-Tempered MetaDynamics（~2–3 天）
11. Phase 9: 自由能面重构（DRAFT）（~1 小时）
12. 附录 A: 参数参考表
13. 附录 B: 常见问题排查

前置条件

开始之前你需要什么

本教程不假设你已经安装了任何软件。你只需要：

- **一台有终端（terminal）的电脑** —macOS 自带 Terminal.app，Windows 可以用 WSL 或 PuTTY，Linux 自带终端
- **HPC 集群账号** —下面会说怎么申请
- **网络连接** —需要 SSH 到集群、下载 PDB 文件等

如何获取 HPC 访问权限

分子动力学模拟需要 GPU 加速的高性能计算集群（HPC cluster）。你需要先申请一个账号。

UNC Longleaf（本教程使用的集群）：

- 去 https://help.rc.unc.edu/getting-started/ 按指引申请
- 需要你的导师（PI）支持、一段简短的项目描述、以及你的 UNC ONYEN

其他学校：

- 搜索“[你的学校名] HPC”或“[你的学校名] research computing”
- 大多数研究型大学都有免费或低成本的 HPC 资源
- 申请通常需要：导师签字、项目描述（1–2 句话即可）、学校 ID

GPU 访问：GPU 节点（用于 `pmemd.cuda` 跑 MD）可能需要额外申请。申请时说明你需要 NVIDIA GPU 做分子动力学模拟即可。

如何通过 SSH 连接到集群

SSH（Secure Shell）是你从自己电脑连接到远程集群的方式。所有后续操作都在集群上执行。

基本连接命令：

> **终端基础知识**：以 # 开头的行是注释，计算机会忽略它们，只是写给你看的说明。没有 # 前缀的行是你需要输入的命令。$ 符号代表终端提示符，不需要输入它，只需输入它后面的内容。

```
ssh your_username@login.hpc.your_university.edu
# UNC Longleaf 示例:
ssh your_onyen@longleaf.unc.edu
```

配置 SSH 快捷方式（推荐，省去每次输入完整地址）：

在你**本地电脑**上编辑 `~/.ssh/config` 文件：

> **什么是 `cat > file << 'EOF'`？** 这叫“heredoc”（嵌入文档）——它直接在终端里创建文件。在 `<< 'EOF'` 和结尾单独一行的 `EOF` 之间的所有内容都会被写入文件。效果等同于打开文本编辑器、输入内容、保存。我们用这种方式是为了让你能一次性复制粘贴整个代码块。
>
> **> 和 >> 的区别**：> 创建新文件（如果文件已存在则覆盖）。>> 在现有文件末尾追加内容，不会删除原有内容。> = “从头写”，>> = “接着写”。

```
# 在本地电脑上运行，不是在集群上
mkdir -p ~/.ssh
```

```
cat >> ~/.ssh/config << 'EOF'

Host longleaf
    HostName longleaf.unc.edu
    User your_onyen
    ControlMaster auto
    ControlPath ~/.ssh/sockets/%r@%h-%p
    ControlPersist 600
EOF

mkdir -p ~/.ssh/sockets
```

配置后，只需输入 `ssh longleaf` 即可连接。`ControlMaster` 配置让你打开多个终端窗口时不需要重复输入密码。

在本地和集群之间传输文件：

```
# 从本地上传到集群
scp local_file.txt longleaf:/work/users/x/xx/your_onyen/

# 从集群下载到本地
scp longleaf:/work/users/x/xx/your_onyen/remote_file.txt ./
```

软件概览

本项目需要以下软件。不用担心——Phase 0 会一步步带你安装和验证每一个。

软件	是什么	从哪里获取	是否免费
AMBER	分子动力学模拟引擎（经典 MD 部分使用 `pmemd.cuda` GPU 加速）	HPC 通常已装：`module load amber`；购买：https://ambermd.org/	商业软件（学校通常有 license）
AmberTools	力场参数化工具集（`antechamber`, `tleap`, `parmchk2` 等）	免费下载：https://ambermd.org/AmberTools.php	免费
Gaussian	量子力学（QM）计算，用于 RESP 电荷拟合	HPC 通常已装：`module load gaussian`；需要许可证	商业软件
GROMACS	MetaDynamics 模拟引擎（需 PLUMED 支持）	conda 安装（Phase 0 会教）；官网：https://www.gromacs.org/	免费开源
PLUMED	增强采样插件（enhanced sampling），定义 CV 和偏置势	conda 安装（Phase 0 会教）；官网：https://www.plumed.org/	免费开源
conda	软件包管理器，用于安装 GROMACS 和 PLUMED	HPC 通常已装：`module load anaconda`；或安装 Miniconda：https://docs.conda.io/en/latest/miniconda.html	免费

软件	是什么	从哪里获取	是否免费
Python	分析脚本	conda 环境自带	免费

什么是力场（Force field）？ 力场是一组描述原子如何相互作用的数学方程和参数——键怎么伸缩、角度怎么弯曲、带电原子怎么吸引或排斥。ff14SB 是专门为蛋白质设计的一套参数。

没有 Gaussian 怎么办？ Gaussian 是商业软件，不是所有学校都有。免费替代方案：- **ORCA** (https://www.faccts.de/orca/)：学术免费，支持 RESP 电荷计算 - **Psi4** (https://psicode.org/)：完全开源，`conda install -c conda-forge psi4`

使用替代软件时，QM 计算的输入文件格式不同，但 RESP 拟合的下游流程（`antechamber` 读取输出）是一样的。

磁盘空间需求

内容	大小	说明
PDB 文件 + 参数文件	< 100 MB	结构、力场参数
RESP 电荷计算	~1 GB	Gaussian 临时文件较大
经典 MD 轨迹（500 ns）	~20 GB	`.nc` 格式的轨迹文件
MetaDynamics 轨迹	~20 GB	GROMACS `.xtc` 格式
分析输出	~2 GB	FES、图表等
总计	**~45 GB**	建议预留 50 GB

大多数 HPC 集群的 home 目录空间有限（如 Longleaf 只有 50 GB）。把大文件放在 work/scratch 目录下。

关键参考文献

- **SI pp. S2–S3**：体系搭建、力场（force field）、MD 方案
- **SI pp. S3–S4**：MetaDynamics 参数、path CV 构建
- **Caulkins et al.**[1]：PLP 质子化状态（protonation states）（净电荷 = -2）
- **Wang et al.**[2]：GAFF 力场（force field）
- **Bayly et al.**[3]：RESP 电荷拟合（charge fitting）

Phase 0: 环境搭建（~30 分钟）

目标：在 HPC 集群上连接、创建工作目录、安装并验证所有软件依赖。

[1] Caulkins, S. G. et al. NMR crystallography of a carbanionic intermediate in tryptophan synthase: chemical structure, tautomers, and reaction specificity. *J. Am. Chem. Soc.* **2014**, *136*, 12824–12827.

[2] Wang, J.; Wolf, R. M.; Caldwell, J. W.; Kollman, P. A.; Case, D. A. Development and testing of a general Amber force field. *J. Comput. Chem.* **2004**, *25*, 1157–1174.

[3] Bayly, C. I.; Cieplak, P.; Cornell, W. D.; Kollman, P. A. A well-behaved electrostatic potential based method using charge restraints for deriving atomic charges: the RESP model. *J. Phys. Chem.* **1993**, *97*, 10269–10280.

Step 0.0: SSH 连接到集群

如果你按照上面的 `~/.ssh/config` 配置过了，直接运行：

```
ssh longleaf
```

否则用完整命令：

```
ssh your_onyen@longleaf.unc.edu
```

连接成功后你会看到集群的欢迎信息和命令行提示符。后面所有命令都在**集群上**运行。

Step 0.1: 创建项目目录结构

> 目录名称可以随意取——`AnimaLab`、`structures` 等只是本教程的约定，不是硬性要求。你可以用任何你喜欢的名称。重要的是保持一致的组织结构。

```
# 先设置 WORK 变量（替换为你自己的路径）
export WORK=/work/users/<首字母>/<前两个字母>/<你的 ONYEN>/AnimaLab
# 例如: export WORK=/work/users/j/jo/johndoe/AnimaLab

# 创建目录结构
# 行末的 \ 表示命令在下一行继续（见下面的说明）
mkdir -p $WORK/{structures,parameters/{plp_structures,resp_charges,mol2, \
  frcmod},systems/pftrps_ain,scripts,runs/pftrps_ain_md,logs, \
  analysis}
```

> **行末的 \ 是什么？** 反斜杠告诉 shell “这个命令在下一行继续”。用来把很长的命令分成多行，方便阅读。复制粘贴时，shell 会把所有 \ 连接的行当作一条命令执行。

每个目录的用途：

目录	存放什么
`structures/`	PDB 晶体结构文件
`parameters/`	力场参数（PLP 的 mol2、frcmod 文件）
`systems/`	tleap 构建的完整体系（prmtop + inpcrd）
`scripts/`	运行脚本、PLUMED 输入文件
`runs/`	生产运行（production run）的输出目录
`logs/`	Slurm 作业日志
`analysis/`	自由能面（FES）、轨迹分析结果

验证：

```
ls -d $WORK/*/
# 应列出: analysis, logs, parameters, runs, scripts, structures, systems
```

Step 0.2: 检查 HPC 已有软件

HPC 集群通常预装了很多科学软件，通过 `module` 系统管理。先看看有什么可以直接用：

什么是 | (管道)? 竖杠 | 把前一个命令的输出传给下一个命令作为输入。可以理解为流水线：命令 1 | 命令 2 意思是"运行命令 1，然后把结果喂给命令 2 处理"。比如 `cat file.txt | grep "hello"` 先读取文件，再从中搜索包含"hello"的行。

什么是 head 和 tail? `head -N` 显示文件的前 N 行，`tail -N` 显示最后 N 行。用来快速预览大文件而不需要全部打开。`tail -1` 只显示最后一行。

什么是 2>&1? 程序有两种输出：正常输出（stdout，编号 1）和错误信息（stderr，编号 2）。2>&1 把错误信息合并到正常输出中，这样两种输出都会一起显示或保存到日志文件里。配合 `| tee logfile` 可以同时在屏幕上看到输出并保存到文件。

```
# 列出所有可用的 module（输出可能很长，用 grep 过滤）
module avail amber 2>&1 | head -20
module avail gaussian 2>&1 | head -20
module avail anaconda 2>&1 | head -20
module avail gromacs 2>&1 | head -20
```

什么是 module load? HPC 集群装了很多软件，但默认不全部激活（避免版本冲突）。`module load amber/24p3` 激活 AMBER 24p3 版本，把它的程序加入你的 PATH，这样你就能使用了。`module avail` 列出所有可用软件。

如果某个软件找不到：

- AMBER/Gaussian：联系你的 HPC 管理员（help@rc.unc.edu for Longleaf），问他们是否有安装以及如何访问
- GROMACS/PLUMED：不用担心，下一步我们用 conda 安装带 PLUMED 支持的版本
- conda/Anaconda：如果 `module avail anaconda` 没有结果，你可以自行安装 Miniconda：

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p $HOME/miniconda3
eval "$($HOME/miniconda3/bin/conda shell.bash hook)"
```

Step 0.3: 用 conda 安装 PLUMED + GROMACS

什么是 conda-forge? conda-forge 是一个社区维护的 conda 软件仓库（channel），里面有很多科学计算软件的预编译版本。我们从这里安装 GROMACS 是因为 conda-forge 的版本在编译时已经打了 PLUMED 补丁（patch），可以直接使用 PLUMED 的增强采样功能。HPC 系统自带的 GROMACS 通常没有这个补丁。

什么是 PLUMED? PLUMED 是一个增强采样（enhanced sampling）插件。它作为 GROMACS 的插件运行，在模拟过程中施加偏置势（bias potential），推动体系探索更多的构象空间。MetaDynamics 就是 PLUMED 实现的一种增强采样方法。

什么是 GROMACS? GROMACS 是一个高性能分子动力学引擎。本项目中，经典 MD 用 AMBER 跑（因为我们用 AMBER 的力场工具链），但 MetaDynamics 必须用 GROMACS + PLUMED（因为原文就是这么做的，我们严格复刻）。

```
cat > $WORK/scripts/setup_plumed_longleaf.sh << 'EOF'
#!/bin/bash
# Purpose: Create conda environment with PLUMED 2.9 + GROMACS (PLUMED-patched)
# Why conda: Longleaf's system GROMACS has no PLUMED support; conda-forge do.
```

```
#   builds
# Source: [operational default]

module load anaconda/2024.02
module load amber/24p3

# Create environment
conda create -n trpb-md python=3.10 -y

# Activate (must use eval for non-interactive shell)
# $() 运行括号里的命令并把输出替换进来（见下面的说明）
eval "$(conda shell.bash hook)"
conda activate trpb-md

# Install PLUMED (includes GROMACS with PLUMED support)
# Why conda-forge: pre-built with PLUMED patch applied at compile time
conda install -c conda-forge plumed py-plumed gromacs -y

# Verify installations
echo "=== PLUMED ==="
plumed --version
# Expected: 2.9.x

echo "=== GROMACS ===="
gmx --version | head -5
# Expected: GROMACS 2026.0

echo "=== AMBER ===="
which pmemd.cuda
# Expected: /nas/longleaf/apps/amber/24p3/.../pmemd.cuda
EOF

chmod +x $WORK/scripts/setup_plumed_longleaf.sh
```

> **什么是 `$()`?** 它运行括号里的命令，然后把输出结果替换进来。比如 `echo " 今天是 $(date)"` 会先运行 `date` 获取日期，再插入到 echo 命令中。
>
> **什么是 `chmod +x`?** `chmod` 修改文件权限。`+x` 让文件变成"可执行"的——也就是可以当作程序运行。不加这个的话，系统会拒绝运行脚本。
>
> **如果找不到模块**：运行 `module avail amber` 或 `module avail gaussian` 查看集群上可用的版本。模块名称因集群而异。如果完全没有该软件，联系 HPC 管理员。
>
> **如果 `conda` 命令找不到**：参见 Step 0.2 安装 Miniconda 的说明。安装后需要重新登录或运行 `source ~/.bashrc`。
>
> **如果 `conda` 安装卡住**：尝试使用更快的求解器：`conda install ... --solver=libmamba`。或者先安装 libmamba-solver：`conda install -n base conda-libmamba-solver`。

运行脚本（安装过程大约需要 5–10 分钟）：

```
bash $WORK/scripts/setup_plumed_longleaf.sh
```

验证：

> **什么是 &&?** 意思是"只有前一个命令成功了，才执行下一个命令"。命令 1 && 命令 2 如果命令 1 失败了，命令 2 就不会执行。

```
module load anaconda/2024.02 && conda activate trpb-md
plumed --version        # 应输出 2.9.x
gmx --version 2>&1 | grep "GROMACS version"  # 应输出 2026.0
```

Step 0.4: 验证 AMBER / AmberTools

AMBER 包含两部分：商业版的 `pmemd`（MD 引擎，GPU 加速）和免费的 AmberTools（`antechamber`, `tleap`, `parmchk2` 等参数化工具）。

```
module load amber/24p3

# 验证 MD 引擎
which pmemd.cuda
# Expected: /nas/longleaf/apps/amber/24p3/.../pmemd.cuda

# 验证参数化工具
which antechamber       # 应输出有效路径
which tleap             # 应输出有效路径
which parmchk2          # 应输出有效路径

# 检查版本
antechamber --version 2>&1 | head -3
```

> **如果 AMBER 不可用**：AmberTools 可以免费下载（https://ambermd.org/AmberTools.php）。没有 `pmemd.cuda` 的话，你仍然可以完成参数化步骤，但经典 MD 需要用 `sander`（AmberTools 自带，CPU 版本，慢得多）或改用 GROMACS 跑。

Step 0.5: 验证 Gaussian

Gaussian 用于量子力学（QM）计算，在本项目中用来做 RESP 电荷拟合（charge fitting）——给 PLP 辅因子的每个原子分配部分电荷。

```
module load gaussian/16c02
which g16
# Expected: /nas/longleaf/apps/gaussian/16c02/g16
```

> **没有 Gaussian 怎么办？** Gaussian 是商业授权软件，不是所有学校都有。
>
> - 先检查你的集群：`module avail gaussian`
> - 联系 HPC 管理员询问是否有安装
> - 免费替代方案：
> - **ORCA**（https://www.faccts.de/orca/）：学术免费，需注册。支持 HF/6-31G* 级别的计算，`antechamber` 可以读取 ORCA 的输出

  - **Psi4**（https://psicode.org/）：完全开源，可通过 conda 安装：

    ```
    conda install -c conda-forge psi4 -y
    ```

- 使用替代软件时，QM 计算的输入格式不同，但下游 RESP 拟合流程（antechamber 读取 QM 输出 □ 生成 mol2）是一样的

Step 0.6: 保存环境配置到 ~/.bashrc

为了避免每次登录都手动加载 module，把常用配置写入 ~/.bashrc：

```
cat >> ~/.bashrc << 'EOF'

# === TrpB MetaDynamics project ===
module load amber/24p3
module load anaconda/2024.02
# conda activate trpb-md  # 取消注释以自动激活 conda 环境
export WORK=/work/users/< 首字母 >/< 前两个字母 >/< 你的 ONYEN>/AnimaLab
EOF
```

> **注意**：把 < 你的 ONYEN> 替换为你的实际用户名。conda activate 那行默认注释掉了——如果你希望每次登录自动进入 conda 环境，取消注释即可。

让配置立即生效：

```
source ~/.bashrc
```

最终验证——确认所有软件就绪：

```
echo "=== AMBER ===" && which pmemd.cuda && which antechamber
echo "=== conda ===" && conda --version
echo "=== PLUMED ===" && conda activate trpb-md && plumed --version
echo "=== GROMACS ===" && gmx --version 2>&1 | grep "GROMACS version"
echo "=== Gaussian ===" && which g16 2>/dev/null \
  || echo "Not available (see alternatives above)"
echo "=== WORK dir ===" && ls -d $WORK/*/
```

如果以上命令都输出了有效路径/版本号，恭喜——你的环境已经准备好了。

Phase 1: 下载与准备 PDB 结构（~10 分钟）

> **目标**：获取 Ain 中间体（intermediate）的晶体结构以及 path CV 端点结构。

背景

JACS 2019 研究使用了三个 PDB 结构：

> **什么是残基（Residue）?** 残基是蛋白质链的一个组成单元。在蛋白质中，每个残基就是一个氨基酸（如丙氨酸、甘氨酸、赖氨酸等）。非标准残基（如我们的 PLP-赖氨酸）需要特殊处理。

PDB	用途	关键残基	来源
5DVZ	Ain（内部醛亚胺，internal aldimine）参数化	LLP（24 个重原子，chain A）	[SI p.S2]
1WDW	path CV 的 Open 态端点	—	[SI p.S3]
3CEP	path CV 的 Closed 态端点	—	[SI p.S3]

PLP 和 LLP 到底是什么关系?

这是一个非常容易搞混的地方，简单说：

- **PLP**（pyridoxal 5'-phosphate，磷酸吡哆醛）是**游离态的辅因子**——就是这个小分子本身，没有连接到任何东西上。可以把它想象成一把放在桌上的钥匙。
- **LLP** 是 PDB 数据库给 PLP **共价连接到赖氨酸残基上**之后取的名字（在我们的体系中是 Lys82）。连接方式是席夫碱（Schiff base，一个 C=N 双键）。可以把它想象成钥匙插进了锁里——它和游离的钥匙在化学上已经不同了，因为形成了新的共价键。（**席夫碱**是醛基和氨基反应形成的 C=N 双键。在 TrpB 中，PLP 的醛基与 Lys82 的氨基反应形成这个键，把辅因子共价连接到蛋白质上。）

在晶体结构 5DVZ 中，PLP 不是游离的——它已经共价连接到了 Lys82 上。所以 PDB 把它注册为 **LLP**（全名：N6-(pyridoxal phosphate)-L-lysine），而不是 PLP。如果你在 5DVZ 的 PDB 文件里搜索"PLP"，什么都找不到。残基名就是 **LLP**。

在本教程中：- 我们说"PLP"时，是在**泛指**这个辅因子（比如"PLP 参数化"）- 我们用"LLP"时，是指 PDB 文件中的**具体残基名** - 后面 antechamber 会把它重命名为"AIN"（Ain 中间体的缩写）——这只是标签变化，不影响化学结构

一句话总结：PLP、LLP、AIN 在不同语境下指的是同一个分子。原子和电荷完全一样。

Step 1.1: 下载 PDB 文件

```
cd $WORK/structures

wget -O 5DVZ.pdb https://files.rcsb.org/download/5DVZ.pdb

# 验证下载成功:
ls -lh 5DVZ.pdb
# 预期: 文件大小不为零
# 如果下载失败（文件为 0 字节或内容是 "404 Not Found"）:
#   - 检查 URL 是否正确
#   - 尝试: curl -L https://files.rcsb.org/download/5DVZ.pdb -o 5DVZ.pdb
#   - 检查网络连接
wget -O 1WDW.pdb https://files.rcsb.org/download/1WDW.pdb

# 验证下载成功:
ls -lh 1WDW.pdb
```

```
# 预期：文件大小不为零
# 如果下载失败（文件为 0 字节或内容是 "404 Not Found"）：
#   - 检查 URL 是否正确
#   - 尝试：curl -L https://files.rcsb.org/download/1WDW.pdb -o 1WDW.pdb
#   - 检查网络连接
wget -O 3CEP.pdb https://files.rcsb.org/download/3CEP.pdb

# 验证下载成功：
ls -lh 3CEP.pdb
# 预期：文件大小不为零
# 如果下载失败（文件为 0 字节或内容是 "404 Not Found"）：
#   - 检查 URL 是否正确
#   - 尝试：curl -L https://files.rcsb.org/download/3CEP.pdb -o 3CEP.pdb
#   - 检查网络连接
```

什么是 grep？ grep 在文件或命令输出中搜索文本。grep "hello" file.txt 找到所有包含"hello"的行。-c 表示只计数不显示，-q 表示静默检查（用在脚本里判断是否匹配）。

什么是 awk？ awk 是按列处理文本的工具。PDB 文件中每一行都是固定宽度的格式（原子名在第 13-16 列，残基名在第 18-20 列，链 ID 在第 22 列等）。awk 可以提取指定列。比如 substr($0,18,3) 表示从第 18 个字符开始取 3 个字符——在 PDB 格式中这就是残基名。

什么是 wc？ wc 是"word count"（字数统计）的缩写。wc -l 统计文件的行数。我们用它来验证文件是否包含预期数量的条目。

验证：

```
ls -lh $WORK/structures/*.pdb
# 应显示三个 PDB 文件，每个 > 100 KB

# 确认 5DVZ 中含有 LLP 残基：
grep "^HETATM" $WORK/structures/5DVZ.pdb \
  | awk '{print substr($0,18,3)}' \
  | sort -u
# 应包含：LLP
```

什么是 sort -u？ sort 按字母排序。-u 参数只保留唯一行（去重）。合在一起，sort -u 给你一个去重后的列表。

什么是 grep -E？ -E 参数启用"扩展正则表达式"。^(ATOM |HETATM) 表示"以 ATOM 或 HETATM 开头的行"——这是 PDB 文件中包含原子坐标的两种记录类型。^ 表示"行首"，| 表示"或"。

awk 中的 gsub 是什么？ gsub(/模式/, " 替换", 变量) 做全局替换——把变量中所有匹配模式的内容替换掉。gsub(/ /,"",rn) 的意思是删除变量 rn 中的所有空格。

```
# 统计 chain A 中 LLP 的重原子数：
grep -E "^(HETATM|ATOM  )" $WORK/structures/5DVZ.pdb \
  | awk '{rn=substr($0,18,3); gsub(/ /,"",rn); \
    cid=substr($0,22,1); \
    if(rn=="LLP" && cid=="A") print}' \
```

```
  | wc -l
# Expected: 24
```

Step 1.2: 从 5DVZ 提取 chain A

PDB 文件格式说明：PDB 文件存储蛋白质中每个原子的三维坐标。每一行代表一个原子，使用固定列宽格式：- 第 1-6 列：记录类型（ATOM = 蛋白质原子，HETATM = 配体/辅因子）- 第 13-16 列：原子名（如 CA = α 碳，NZ = ζ 氮）- 第 18-20 列：残基名（如 ALA = 丙氨酸，LLP = PLP-赖氨酸）- **第 22 列：链 ID**（Chain ID）——一个字母，标识这个原子属于哪条蛋白链 - 第 23-26 列：残基编号 - 第 31-54 列：X, Y, Z 坐标（单位：埃）

为什么选 Chain A? 晶体结构通常包含同一蛋白的多个拷贝。PDB 5DVZ 有 4 条链（A、B、C、D），都是 TrpB 的完全相同的拷贝。我们任选 Chain A——选哪条结果都一样。链 ID 在每行的第 22 列。

为什么：5DVZ 有 4 条链（A、B、C、D）。benchmark 体系只需要 chain A（独立的 beta 亚基）。[SI p.S2]

```
grep -E "^(ATOM  |HETATM)" $WORK/structures/5DVZ.pdb \
  | awk '{cid=substr($0,22,1); if(cid=="A") print}' \
  > $WORK/structures/5DVZ_chainA.pdb
```

验证：

```
wc -l $WORK/structures/5DVZ_chainA.pdb
# 应为 ~2800-3200 行（一条 ~390 残基蛋白质的链 + LLP）

grep "LLP" $WORK/structures/5DVZ_chainA.pdb | wc -l
# Expected: 24
```

Step 1.3: 为 tleap 准备蛋白质 PDB

做什么：将 LLP 残基重命名为 AIN（我们内部对 Ain 中间体的命名），并清理 PDB 以便 AMBER 处理。

为什么：antechamber 输出的 mol2 文件中残基名称为“AIN”（来源于输出文件名，参见 FP-017）。我们必须让 PDB 和 mol2 的残基名称一致，这样 tleap 才能正确识别辅因子（cofactor）。

命名流程：PDB 文件里是“LLP”□ 我们在 PDB 中把它改名为“AIN”□ antechamber 输出的 mol2 里也是“AIN”□ 两个文件名字一致 □ tleap 才能把它们对应起来。如果名字不一致，tleap 就认不出辅因子，会报错。

什么是 `grep -v`? -v 参数表示**反向匹配**——显示不包含指定文本的行。所以 `grep -v "HOH"` 会删除所有包含“HOH”（水分子）的行。可以理解为”排除”。我们连续用两个 `grep -v` 来同时去掉水分子和交替构象。

```
cd $WORK/structures

# 移除水分子、离子和交替构象；将 LLP 重命名为 AIN
grep -E "^(ATOM  |HETATM)" 5DVZ_chainA.pdb \
  | grep -v "HOH" \
```

```
  | grep -v " B " \
  | sed 's/LLP/AIN/g' \
  > $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb
```

验证：

```
grep "AIN" $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb | wc -l
# Expected: 24（重命名后的 LLP 原子）

grep "LLP" $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb | wc -l
# Expected: 0（全部已重命名）
```

Phase 2: PLP 辅因子参数化（~2–3 天）

目标：为 PLP-K82 Schiff 碱辅因子（Ain 中间体）生成 AMBER 兼容的力场参数（GAFF 原子类型 + RESP 电荷）。

为什么需要这一步：ff14SB 力场（force field）只包含 20 种标准氨基酸的参数。LLP（磷酸吡哆醛通过 Schiff 碱与 Lys82 共价连接）是非标准残基，必须单独参数化。SI 指定使用 GAFF 原子类型，在 HF/6-31G(d) 水平上拟合 RESP 电荷 [SI p.S2]。

这是整个 pipeline 中最复杂的阶段。下面对每个子步骤进行详细说明。

RESP 参数化工作流概览

```
PDB (5DVZ, LLP residue)
  |
  v
[Step 2.1] Extract LLP fragment from PDB
  |         Purpose: isolate the non-standard residue
  v
[Step 2.2] Add hydrogens (reduce)
  |         Purpose: antechamber needs H to determine bond orders
  v
[Step 2.3] Add ACE/NME capping groups
  |         Purpose: neutralize dangling peptide bonds for QM
  v
[Step 2.4] Generate Gaussian input (antechamber)
  |         Purpose: set up QM geometry optimization + ESP calculation
  v
[Step 2.5] Run Gaussian 16 QM calculation (~12-48 hrs)
  |         Purpose: compute electrostatic potential around molecule
  v
[Step 2.6] Extract RESP charges (antechamber)
  |         Purpose: fit point charges to reproduce QM ESP
  v
```

```
[Step 2.7] Generate missing parameters (parmchk2)
  |         Purpose: estimate bonded terms not in GAFF
  v
[Step 2.8] Validate output files
          Purpose: ensure charges, atom types, and connectivity are correct
```

Step 2.1: 从 5DVZ 提取 LLP 片段

做什么：从 5DVZ 的 chain A 中提取 LLP 残基的 24 个重原子（heavy atoms）。

为什么：我们需要一个独立的非标准残基片段，用于量子力学电荷计算。蛋白质的其余部分将使用 ff14SB 参数。

```
cd $WORK/parameters/plp_structures

grep -E "^(HETATM|ATOM  )" $WORK/structures/5DVZ.pdb \
  | awk '{rn=substr($0,18,3); gsub(/ /,"",rn); \
    cid=substr($0,22,1); \
    if(rn=="LLP" && cid=="A") print}' \
  > $WORK/parameters/plp_structures/LLP_chainA.pdb
```

验证：

```
wc -l $WORK/parameters/plp_structures/LLP_chainA.pdb
# Expected: 24 行（24 个重原子）

# 打印原子名称以验证完整性：
awk '{printf "%s\n", substr($0,13,4)}' \
  $WORK/parameters/plp_structures/LLP_chainA.pdb
# 应包含主链原子（N, CA, C, O）和 PLP 环原子（C1, C2, C3 等）
```

Step 2.2: 用 reduce 添加氢原子

做什么：使用 reduce 程序（AmberTools 自带）为提取的 LLP 片段添加氢原子。

为什么：antechamber 需要氢原子才能正确判断键级（bond orders）和芳香性（aromaticity）。如果没有氢原子，吡啶环（pyridine ring）原子会被错误地分配为 c1（sp 杂化）而不是 ca（芳香型），导致力场参数错误。此问题已记录为 failure pattern FP-010。

reduce 的作用：reduce 利用标准键长、范德华半径和氢键模式，在化学上合理的位置放置氢原子。它处理：(1) 主链酰胺 NH，(2) 芳香 CH，(3) 甲基 CH3，(4) 羟基 OH。

```
module load amber/24p3

reduce $WORK/parameters/plp_structures/LLP_chainA.pdb \
  > $WORK/parameters/plp_structures/LLP_chainA_H.pdb
```

验证：

```
wc -l $WORK/parameters/plp_structures/LLP_chainA_H.pdb
# Expected: ~42 行（24 个重原子 + ~18 个氢原子）
```

```
# 统计氢原子数:
grep " H" $WORK/parameters/plp_structures/LLP_chainA_H.pdb | wc -l
# Expected: ~18 个氢原子
```

Step 2.3: 添加 ACE/NME 封端基团（capping groups）

做什么：在 LLP 片段的主链 N 端和 C 端分别连接乙酰基（ACE）和 N-甲基胺基（NME）封端基团。

为什么：将 LLP 从蛋白质中切出后，主链 N 和 C 原子处会留下悬挂的肽键（dangling peptide bonds）。实际上，这些原子分别与相邻残基（Gly81 和 Ser83）连接。如果不加封端，量子力学计算会看到非物理的开放化合价（open valences），导致末端电荷失真。封端基团模拟了蛋白质主链的电子环境。

ACE 封端 (CH3-CO-)：模拟前一个残基的羰基。连接到主链 N 原子上。**NME 封端** (NH-CH3)：模拟后一个残基的酰胺基。连接到主链 C 原子上。

注意：这一步需要手动操作。推荐使用分子编辑器(PyMOL Builder 或 UCSF Chimera)添加封端并保存结果。也可以使用为本项目开发并验证过的 Python 脚本 `build_llp_ain_capped_resp.py`。

```
# The capped structure should have:
#   - 17 carbon atoms
#   - 4 nitrogen atoms
#   - 7 oxygen atoms
#   - 1 phosphorus atom
#   - 25 hydrogen atoms
#   Total: 54 atoms
#
# Z_total (sum of atomic numbers) = 226 (MUST be even)
# Net charge = -2   [Caulkins 2014]
# Electrons = 226 - (-2) = 228 (even -> singlet OK)
#
# FP-012 guard: (Z_total - charge) must be even for singlet multiplicity
```

封端完成后（无论用哪种方法），保存结果：

```
ls -lh $WORK/parameters/plp_structures/LLP_ain_capped.pdb
# Expected: file exists, ~54 atoms
```

验证：

```
# 统计封端结构的总原子数:
grep -c "^ATOM\|^HETATM" $WORK/parameters/plp_structures/LLP_ain_capped.pdb
# Expected: 54

# 验证电子数奇偶性（关键检查，参见 FP-012）:
# Z_total = 6*17 + 7*4 + 8*7 + 15*1 + 1*25 = 102 + 28 + 56 + 15 + 25 = 226
# Electrons = 226 - (-2) = 228（偶数）-> singlet (mult=1) 有效
```

Step 2.4: 准备 Gaussian 输入文件用于 RESP ESP 计算

做什么：创建一个 Gaussian 16 输入文件，执行 (1) 几何优化（geometry optimization）和 (2) Merz-Kollman 静电势（ESP）计算。

为什么：RESP 电荷通过拟合点电荷来重现量子力学计算的静电势。HF/6-31G(d) 是 RESP 拟合的标准理论水平，因为它系统性地高估偶极矩约 10-20%，这部分补偿了 GAFF 等固定电荷力场缺乏显式极化的不足。[SI p.S2]

为什么用 RESP 而不是 AM1-BCC? AM1-BCC 更快，但使用半经验方法，对 PLP 不收敛——PLP 的复杂电子结构（共轭芳香环 + 磷酸基 + 亚胺）让 SCF 计算发散。RESP 使用完整的量子力学计算（HF/6-31G(d)），能正确处理 PLP，代价是需要 Gaussian 软件。此问题已记录为 FP-009。

Gaussian route line 各关键字说明：

Keyword	用途
`HF/6-31G*`	Hartree-Fock 理论 + 6-31G(d) 基组。RESP 拟合的标准方法。[SI p.S2]
`SCF=tight`	更严格的 SCF 收敛标准 (10^-8 a.u.)。确保 ESP 可靠。
`Pop=MK`	Merz-Kollman 布居分析（population analysis）：在分子周围的网格点上输出 ESP。RESP 拟合读取这些数据。
`iop(6/33=2)`	设置 ESP 网格点的同心层数为 2。更多层 = 更好的电荷拟合。
`iop(6/42=6)`	设置每层单位面积上的点密度。更高 = 更多 ESP 数据点。
`opt`	几何优化：在 HF/6-31G(d) 水平上将封端结构弛豫到局部最小值，然后再计算 ESP。

关于 iop(6/50=1) 的说明：一些 RESP 教程包含 `iop(6/50=1)` 用于写入单独的 `.gesp` 文件。我们**不使用**这个选项，因为它会在 Gaussian 16c02 中导致“Blank file name read”错误（记录为 FP-013）。取而代之，antechamber 通过 `-fi gout` 标志直接从 Gaussian `.log` 文件读取 ESP 数据（参见 FP-014）。

净电荷分解（Ain/LLP）

来源：Caulkins et al.[4]（PLP Schiff 碱的固态 NMR）

官能团	电荷	理由
磷酸基团（Phosphate group, PO4）	-2	生理 pH 下三个脱质子化的氧
吡啶鎓 N1（Pyridinium N1）	+1	质子化的环氮（pKa ~5.5，pH 7.8 时质子化）
酚氧负离子 O3（Phenolate O3）	-1	脱质子化的酚氧（pKa ~3-4）
Schiff 碱 NZ（亚胺，imine）	0	Ain 态下的中性亚胺连接

[4] Caulkins, S. G. et al. NMR crystallography of a carbanionic intermediate in tryptophan synthase: chemical structure, tautomers, and reaction specificity. *J. Am. Chem. Soc.* **2014**, *136*, 12824–12827.

官能团	电荷	理由
ACE/NME 封端	0	设计上电荷中性
净电荷	**-2**	-2 + 1 + (-1) + 0 + 0 = **-2**

```
cat > $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt << 'EOF'
%chk=LLP_ain_resp_capped.chk
%nprocshared=8
%mem=8GB
#HF/6-31G* SCF=tight Pop=MK iop(6/33=2) iop(6/42=6) opt

ACE-LLP-NME (Ain internal aldimine) for RESP charge fitting; charge=-2 [Caulkins 2014]

-2 1
[COORDINATES GO HERE]
[Paste Cartesian coordinates from the capped PDB, one atom per line]
[Format: Element   X.xxxxxx   Y.yyyyyy   Z.zzzzzz]


EOF
```

生成 Gaussian 输入文件

> **实际操作中**，坐标由 `build_llp_ain_capped_resp.py` 脚本生成，该脚本读取封端后的 PDB 并正确格式化坐标。脚本还会验证电子数奇偶性（FP-012 guard）。

验证：

```
head -6 $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt
# 应显示：%chk、%nprocshared=8、%mem=8GB、#HF/6-31G* route line、标题、电荷/多重度

grep -c "^[A-Z]" $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt
# Expected: ~54（坐标块中每个原子一行）
```

Step 2.5: 提交 Gaussian QM 计算

> **做什么**：在 Longleaf 上运行 Gaussian 16 的几何优化 + ESP 计算。
>
> **为什么**：这是参数化过程中的计算瓶颈。对 54 个原子的体系进行 HF/6-31G(d) 计算，使用 8 个核心通常需要 12–48 小时。
>
> **预期输出**：一个 `.log` 文件，包含优化后的几何构型和 Merz-Kollman ESP 数据。文件末尾应有 “Normal termination of Gaussian 16”。
>
> **预计运行时间**：对 ~54 个原子的封端片段做 HF/6-31G(d) 几何优化 +ESP 计算：通常 8 核 CPU 需要 12-48 小时。如果超过 3 天，检查日志文件中是否有 SCF 收敛问题。

```
cat > $WORK/scripts/submit_gaussian_capped.slurm << 'EOF'
#!/bin/bash
#SBATCH --job-name=ain_resp_qm
```

```
#SBATCH --partition=general
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=48:00:00
#SBATCH --output=$WORK/logs/gaussian_ain_resp_%j.out

# Load Gaussian
module load gaussian/16c02

# Set scratch directory (Gaussian writes large temp files)
export GAUSS_SCRDIR=$WORK/logs/gaussian_scratch_${SLURM_JOB_ID}
mkdir -p ${GAUSS_SCRDIR}

cd $WORK/parameters/resp_charges/

echo "Starting Gaussian at $(date)"
g16 LLP_ain_resp_capped.gcrt
echo "Gaussian finished at $(date)"

# Clean up scratch
rm -rf ${GAUSS_SCRDIR}
EOF

sbatch $WORK/scripts/submit_gaussian_capped.slurm
```

> **什么是 sbatch?** 在 HPC 集群上，你不能直接运行耗时的计算——需要提交给作业调度器 (SLURM)。`sbatch script.sh` 把你的脚本提交到队列中。调度器会找到一个有你需要的资源 (GPU、内存等) 的计算节点来运行你的作业。用 `squeue -u $USER` 查看作业状态。
>
> **如果作业立即失败**：检查 Slurm 输出日志：`cat slurm-*.out`。常见原因：(1) 模块未正确加载，(2) 文件路径错误，(3) 请求的资源不可用。用 `squeue -u $USER` 查看作业状态。

验证（任务完成后）：

```
# 检查是否成功终止:
tail -5 $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# 必须包含: "Normal termination of Gaussian 16"

# 检查 ESP 数据是否已写入 (Merz-Kollman charges 部分):
grep -c "Merz-Kollman" $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# Expected: >= 1

# 检查 SCF 收敛问题:
grep "Convergence failure" $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# Expected: (无输出 = 正常)
```

Step 2.6: 用 antechamber 提取 RESP 电荷

做什么：使用 antechamber 读取 Gaussian log 文件，拟合 RESP 点电荷以重现 QM 静电势 (ESP)，同时分配 GAFF 原子类型。

为什么：RESP（Restrained Electrostatic Potential，约束静电势）拟合生成与 AMBER 力场家族兼容的原子偏电荷（partial charges）。拟合过程如下：1. 从 Gaussian log 中读取 Merz-Kollman ESP 网格点 2. 在每个原子上拟合点电荷，使 QM ESP 与点电荷产生的 ESP 之差最小化 3. 对非氢原子施加双曲线约束（将电荷推向零以避免过拟合）4. 采用两阶段拟合：第一阶段约束较弱 (0.0005)，第二阶段对埋藏原子施加更强的约束 (0.001)

antechamber 关键标志说明：- `-fi gout`：读取 Gaussian 输出格式（log 文件）。这会同时提取优化后的几何构型和 ESP 数据。我们使用它代替 `-fi gesp`，因为我们不生成单独的.gesp 文件（参见 FP-013/FP-014）。- `-fo mol2`：输出 Tripos mol2 格式（包含原子类型、电荷和连接信息）- `-c resp`：使用 RESP 电荷拟合方法 [SI p.S2] - `-at gaff`：分配 GAFF (General AMBER Force Field) 原子类型 [SI p.S2] - `-nc -2`：分子净电荷为 -2 [Caulkins 2014] - `-m 1`：多重度为 1（单重态，singlet，所有电子配对）- `-rn AIN`：设置残基名称为 AIN（注意：antechamber 可能会用输出文件名覆盖此设置，参见 FP-017）

```
module load amber/24p3

cd $WORK/parameters

antechamber \
  -i resp_charges/LLP_ain_resp_capped.log \
  -fi gout \
  -o mol2/Ain_gaff.mol2 \
  -fo mol2 \
  -at gaff \
  -c resp \
  -nc -2 \
  -m 1 \
  -rn "AIN" \
  2>&1 | tee $WORK/logs/antechamber_resp_ain.log
```

验证：

```
ls -lh $WORK/parameters/mol2/Ain_gaff.mol2
# 应存在，~3-5 KB

# 检查总电荷是否为 -2：
grep "@<TRIPOS>ATOM" -A 100 $WORK/parameters/mol2/Ain_gaff.mol2 \
  | awk 'NF==9 {sum+=$9} END {printf "Total charge: %.4f\n", sum}'
# Expected: -2.0000（在舍入误差范围内）

# 检查吡啶环的原子类型是否为芳香型：
grep -E "\bca\b|\bnb\b" $WORK/parameters/mol2/Ain_gaff.mol2
# 应显示 ca（芳香碳）和 nb（芳香氮）对应吡啶原子

# 验证 mol2 中的残基名称（FP-017 检查）：
```

```
grep "SUBSTRUCTURE" -A 1 $WORK/parameters/mol2/Ain_gaff.mol2
# 会显示 "AIN"（来自文件名，而非 -rn 标志）
```

Step 2.7: 用 parmchk2 生成缺失的键合参数

做什么：使用 parmchk2 识别 Ain mol2 中标准 GAFF 参数库未覆盖的键（bond）、角（angle）、二面角（dihedral）或非正常扭转（improper torsion）项，并从相似项中估算。（**二面角**由四个原子 A-B-C-D 定义，描述绕 B-C 键的旋转角度，决定分子的三维形状。）

为什么：GAFF 覆盖了大多数常见的有机分子片段，但 PLP 的磷酸基团（P-O 键）和 Schiff 碱连接（C=N-C）具有不常见的 GAFF 原子类型组合。parmchk2 在 GAFF 数据库中搜索最接近的匹配参数并进行外推。penalty score 超过 10 的项应手动审查。

```
parmchk2 \
  -i $WORK/parameters/mol2/Ain_gaff.mol2 \
  -f mol2 \
  -o $WORK/parameters/frcmod/Ain.frcmod
```

验证：

```
ls -lh $WORK/parameters/frcmod/Ain.frcmod
# 应存在，~1-3 KB

# 检查是否有 ATTN 警告（高 penalty 估算的参数）：
grep -c "ATTN" $WORK/parameters/frcmod/Ain.frcmod
# 理想值：0。如果 > 0，审查相关项：
grep "ATTN" $WORK/parameters/frcmod/Ain.frcmod
```

我们的结果：0 个 ATTN 警告（2026-03-31 验证通过，skeptic 6/6 PASS）。

Step 2.8: 后处理—重新分配主链原子类型

做什么：将 K82 主链原子 (C, CA, N, O, CB, CG, CD, CE) 的 GAFF 原子类型替换为其 ff14SB 等价类型，以便 tleap 能与相邻残基正确形成肽键（peptide bonds）。

为什么：antechamber 会对**所有**原子分配 GAFF 类型，包括蛋白质主链（例如 CA 被分配为 c3 而非 ff14SB 的 CT）。当 tleap 构建完整蛋白质时，主链原子需要 ff14SB 类型才能与蛋白质力场匹配。只有 PLP 特有的原子（吡啶环、磷酸基、Schiff 碱）应保留 GAFF 类型。

需要重新分配类型的原子：C8 (backbone C alpha) -> CT, backbone C -> C, backbone N -> N, backbone O -> O, sidechain CB/CG/CD/CE -> CT, sidechain H -> HC/HP

通过编辑 mol2 文件完成此操作。具体的原子名称到类型映射取决于你的 mol2 输出。参见已验证的结果：

```
# Our validated mol2 has 42 atoms with:
# - Backbone retyped to ff14SB: C8(CA)->CT, C(C)->C, N->N, O->O
# - Sidechain retyped: CB,CG,CD,CE -> CT (ff14SB lysine sidechain types)
# - NZ stays GAFF 'nh' (connects to PLP ring)
# - All PLP-specific atoms keep GAFF types (ca, nb, oh, os, p5, o, etc.)
```

验证：

```
# mol2 中的最终原子数:
grep -c "^[[:space:]]*[0-9]" $WORK/parameters/mol2/Ain_gaff.mol2
# Expected: 42（包括封端片段上的氢原子，减去已移除的封端原子）

# 总电荷:
grep "@<TRIPOS>ATOM" -A 100 $WORK/parameters/mol2/Ain_gaff.mol2 \
  | awk 'NF==9 {sum+=$9} END {printf "Total charge: %.4f\n", sum}'
# Expected: -2.0000
```

Phase 2 小结

输出文件	内容	状态
`parameters/mol2/Ain_gaff.mol2`	GAFF 原子类型 + RESP 电荷，42 个原子，charge=-2	已验证（Validated）
`parameters/frcmod/Ain.frcmod`	缺失的键合参数（0 个 ATTN 警告）	已验证（Validated）
`parameters/resp_charges/LLP_ain_resp.log`	Gaussian 16 QM 输出（Normal termination）	已验证（Validated）

Phase 3: 用 tleap 构建体系（~5 分钟）

目标：将参数化后的 Ain 辅因子与蛋白质组合，在 TIP3P 水盒子中溶剂化（solvate），并用抗衡离子（counterions）中和电荷。

什么是抗衡离子（Counterions）? 蛋白质通常带有净电荷。抗衡离子（Na+ 或 Cl-）用来中和这个电荷。没有它们，模拟会产生不真实的长程静电效应。

什么是周期性边界条件? 模拟盒子被自身的无限副本包围。当原子从一侧飞出去，它会从对面重新进入。这个技巧避免了边界效应，让一个小盒子模拟出大系统的效果。

什么是拓扑文件（Topology）? 拓扑文件就像分子系统的蓝图。它列出了每个原子、哪些原子之间有化学键、以及它们之间的相互作用参数（键的弹簧常数、电荷值等）。

tleap 做什么：tleap 是 AMBER 的体系构建工具。它读取力场库文件（ff14SB 用于蛋白质、GAFF 用于辅因子、TIP3P 用于水），加载准备好的 PDB，在周期性盒子中溶剂化，添加反离子，并写出 MD 所需的拓扑文件（topology, .parm7）和坐标文件（.inpcrd）。

Step 3.1: 从 mol2 创建 tleap 库文件

为什么：tleap 需要 `.lib`（或 `.off`）文件来识别 AIN 残基。我们将验证通过的 mol2 转换为此格式。

```
cat > $WORK/scripts/tleap_pftrps_ain.in << 'EOF'
# tleap input for building PfTrpS(Ain) system
# Reference: JACS 2019 SI p.S2 (ff14SB + GAFF + TIP3P + 10 A box + Na+
```

```
#   neutralization)

# Load force fields
source leaprc.protein.ff14SB      # Protein force field [SI p.S2]
source leaprc.gaff                 # GAFF for PLP cofactor [SI p.S2]
source leaprc.water.tip3p          # TIP3P water model [SI p.S2]

# Load Ain (PLP-K82 Schiff base) parameters
AIN = loadmol2 ../parameters/mol2/Ain_gaff.mol2
set AIN head AIN.1.N               # N-terminal connection point (backbone N)
set AIN tail AIN.1.C               # C-terminal connection point (backbone C)
set AIN.1 connect0 AIN.1.N   # Connects to preceding residue
set AIN.1 connect1 AIN.1.C   # Connects to following residue
saveoff AIN ../systems/pftrps_ain/Ain_gaff.lib  # Save as library for reuse

# Load Ain frcmod (missing GAFF parameters)
loadamberparams ../parameters/frcmod/Ain.frcmod

# Reload library (ensures it's registered)
loadoff ../systems/pftrps_ain/Ain_gaff.lib

# Load the prepared protein PDB
# The PDB has LLP renamed to AIN, matching the mol2/lib residue name
prot = loadpdb ../systems/pftrps_ain/5DVZ_chainA_prepared.pdb
check prot

# Solvate in cubic TIP3P box with 10 A buffer [SI p.S2]
solvatebox prot TIP3PBOX 10.0

# Neutralize with Na+ counterions [SI p.S2]
addions prot Na+ 0

# Final check
check prot

# Save AMBER topology and coordinates
saveamberparm prot \
  ../systems/pftrps_ain/pftrps_ain.parm7 \
  ../systems/pftrps_ain/pftrps_ain.inpcrd
savepdb prot ../systems/pftrps_ain/pftrps_ain_leap.pdb
quit
EOF
```

Step 3.2: 运行 tleap

```
module load amber/24p3

cd $WORK/scripts
tleap -f tleap_pftrps_ain.in 2>&1 | tee $WORK/logs/tleap_pftrps_ain.log
```

> **如果 tleap 报 FATAL 错误**：检查 `leap.log` 文件获取详细错误信息。常见原因：(1) 力场文件未加载，(2) 残基名称不匹配，(3) mol2/frcmod 路径错误。运行 `cat leap.log | grep -i "error\|fatal"` 快速定位。

验证：

```
# 检查输出文件是否存在且大小合理：
ls -lh $WORK/systems/pftrps_ain/pftrps_ain.parm7
# Expected: ~600 KB - 1 MB

ls -lh $WORK/systems/pftrps_ain/pftrps_ain.inpcrd
# Expected: ~1-2 MB

# 检查总原子数：
grep -c "^ATOM\|^HETATM" $WORK/systems/pftrps_ain/pftrps_ain_leap.pdb
# Expected: ~39,000（蛋白质 ~6,000 + 水 ~33,000）

# 检查 tleap 日志中的错误：
grep -i "error\|fatal\|warning" $WORK/logs/tleap_pftrps_ain.log
# 审查所有警告。常见的良性警告："close contact" between newly added atoms。

# 检查中和情况：
grep "Na+" $WORK/logs/tleap_pftrps_ain.log
# 应显示添加了 4 个 Na+ 离子（我们的体系中和前净电荷为 -4）

# 检查盒子尺寸：
grep "Box dimensions" $WORK/logs/tleap_pftrps_ain.log
# Expected: 大约 76 x 88 x 73 A
```

> **我们的结果**：总计 39,268 个原子，盒子 76.4 x 88.1 x 73.2 A，4 个 Na+ 离子，净电荷 0。SI 报告 ~15,000 个水分子；我们有 11,092 个（略少，因为 benchmark 只使用了 beta 亚基）。[SI p.S2]

Phase 4: 经典 MD —前处理流程（~24 小时）

> **目标**：通过能量最小化（minimization）、分阶段升温（staged heating）和平衡（equilibration）为产出 MD 准备溶剂化体系。
>
> **什么是能量最小化（Minimization）？** 能量最小化通过调整原子位置来降低系统的势能。想象轻轻摇晃一盒弹珠，让它们落入最低的凹槽——最小化就是对原子做同样的事。它修复初始结构中原子距离过近等问题。
>
> **什么是平衡化（Equilibration）？** 最小化之后，系统能量低了，但还不像真实的物理系统。平衡化

在控制温度和压力的条件下运行短时间的模拟，直到密度和能量等性质稳定下来。

SI 中的方案 [SI pp.S2–S3]：1. 两阶段能量最小化（约束 + 无约束）2. 七步升温（0 -> 350 K，每次升 50 K，NVT 系综，每步 50 ps）3. NPT 平衡（350 K，1 atm，2 ns）

什么是 NVT? NVT 表示原子数（N）、体积（V）、温度（T）保持恒定。模拟盒子大小不变，恒温器维持目标温度。

什么是 NPT? NPT 表示原子数（N）、压力（P）、温度（T）保持恒定。盒子可以膨胀或收缩以维持目标压力（通常 1 atm），让系统达到正确的密度。

参数参考：下面 AMBER 输入文件中的每个参数都在本文档末尾的附录 A中有详细解释。遇到不熟悉的参数，可以先查那里。

Step 4.1: 创建能量最小化输入文件

min1.in —约束最小化

做什么：最小化体系能量，同时固定蛋白质重原子。

为什么：溶剂化后，水分子和离子可能与蛋白质表面存在不利接触。约束最小化在保持晶体结构几何的同时松弛这些接触。500 kcal/mol/A^2 的约束非常强——本质上冻结了蛋白质。[SI p.S2]

```
cat > $WORK/runs/pftrps_ain_md/min1.in << 'EOF'
PfTrpS(Ain) restrained minimization; JACS 2019 SI Conventional MD Minimization Stage 1
&cntrl
  imin=1,
  ntx=1,
  irest=0,
  maxcyc=10000,
  ncyc=5000,
  ntpr=500,
  ntb=1,
  cut=8.0,
  ntr=1,
  restraint_wt=500.0,
  restraintmask='!@H= & !:WAT & !:Na+',
/
EOF
```

参数	值	含义	来源
imin=1	1	运行能量最小化（非 MD）	AMBER manual
ntx=1	1	仅读取坐标（最小化不需要速度）	AMBER manual
irest=0	0	全新开始（非续跑）	AMBER manual
maxcyc=10000	10000	最大 10,000 次最小化循环	[operational default]
ncyc=5000	5000	前 5,000 步使用最陡下降法（robust），之后切换到共轭梯度法（收敛更快）	[operational default]

参数	值	含义	来源
`ntpr=500`	500	每 500 步打印一次能量（监控收敛）	`[operational default]`
`ntb=1`	1	周期性边界条件（periodic boundary conditions），恒体积	AMBER manual
`cut=8.0`	8.0	非键相互作用截断距离 8 A（超过此距离的静电由 PME 处理）	`[SI p.S3]`
`ntr=1`	1	施加位置约束（positional restraints）	`[SI p.S2]`
`restraint_wt=500.0`	500.0	约束力常数：500 kcal/mol/A^2（非常强）	`[SI p.S2]`
`restraintmask`	`'!@H= & !:WAT & !:Na+'`	约束所有非氢、非水、非离子原子（即蛋白质 + 辅因子重原子）	`[SI p.S2]`

什么是 PME（Particle Mesh Ewald）？ PME 是一种快速计算所有原子之间静电相互作用的算法，包括远距离的。没有 PME，大系统的静电计算会慢到不可接受。

AMBER 原子掩码语法：`restraintmask` 用特殊语法选择原子：- `@` 按原子名选择，`:` 按残基名选择 - `!` 表示"不包括"，`&` 表示"并且"- `@H=` 匹配所有氢原子（= 按元素匹配）- 所以 `'!@H= & !:WAT & !:Na+'` 意思是"所有非氢、非水、非钠离子的原子"——即蛋白质和辅因子的重原子

min2.in —无约束最小化

做什么：无约束地最小化整个体系。

为什么：水壳层松弛后（min1），释放所有约束，让整个体系找到局部能量最低点。这解决了剩余的空间位阻冲突（steric clashes）。`[SI p.S2]`

```
cat > $WORK/runs/pftrps_ain_md/min2.in << 'EOF'
PfTrpS(Ain) unrestrained minimization; JACS 2019 SI Conventional MD Minimization Stage
&cntrl
  imin=1,
  ntx=1,
  irest=0,
  maxcyc=10000,
  ncyc=5000,
  ntpr=500,
  ntb=1,
  cut=8.0,
  ntr=0,
/
EOF
```

Step 4.2: 创建升温输入文件（7 步，0 -> 350 K）

做什么：在 7 个 50 ps 的 NVT 步骤中，将体系从 0 K 逐步升温至 350 K，同时逐步降低蛋白质重原子的位置约束。

为什么分七步加热而不是一步到位? 如果直接从 0 K 跳到 350 K，原子会获得巨大的随机速度，导致剧烈碰撞，甚至让模拟崩溃。逐步加热（每次 50 K）让系统在每个温度下先放松，再升高。

为什么约束力逐渐减小? 低温时强约束（210 kcal/mol/A^2）防止蛋白质变形。随着温度升高和系统稳定，约束逐步减弱（210->165->125->85->45->10->10），让蛋白质逐渐获得自然的柔性。

来自 SI 的约束调度表 [SI p.S2] 从 210 kcal/mol/A^2（强）开始，降至 10 kcal/mol/A^2（弱），随着热能增加让蛋白质逐步松弛。

温度 350 K：*P. furiosus* 是一种超嗜热古菌（hyperthermophilic archaeon），最适生长温度为 100 C。该酶在高温下发挥功能；350 K (77 C) 是 SI 中的模拟温度。[SI p.S3]

关于约束调度表的说明：SI 为 7 个升温步骤列出了 6 个约束值（210, 165, 125, 85, 45, 10）。我们对 heat6 和 heat7 都使用 10.0 作为操作选择。[operational default]

```
cat > $WORK/runs/pftrps_ain_md/heat1.in << 'EOF'
PfTrpS(Ain) heating step 1; JACS 2019 SI Heating (50 ps NVT, 0->50 K, 210 kcal/mol/A^2
&cntrl
  imin=0,
  irest=0,
  ntx=1,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  tempi=0.0,
  temp0=50.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=210.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=0.0, VALUE2=50.0,
```

```
/
&wt TYPE='END' /
EOF
```

heat1.in (0 -> 50 K)

参数	值	含义	来源
imin=0	0	运行分子动力学（非最小化）	AMBER manual
irest=0	0	全新开始（heat1 从最小化结构开始）	AMBER manual
ntx=1	1	仅读取坐标（最小化无速度输出）	AMBER manual
nstlim=50000	50000	50,000 步 x 0.001 ps = 每个升温步 50 ps	[SI p.S2]
dt=0.001	0.001	1 fs 时间步长（见下方关于不同 dt 值的说明）	[operational default]
ntc=2	2	SHAKE 约束涉及氢原子的键（见下方说明）	[SI p.S3]
ntf=2	2	不计算 SHAKE 约束键的力（节省计算量）	[SI p.S3]
ntt=3	3	Langevin 恒温器（见下方说明）	[SI p.S3]
gamma_ln=1.0	1.0	Langevin 碰撞频率 (ps^-1)。控制与热浴的耦合强度。	[operational default]
tempi=0.0	0.0	初始温度（仅在 heat1 中，从 0 K 开始）	AMBER manual
temp0=50.0	50.0	本步骤的目标温度	[SI p.S2]
nmropt=1	1	启用 &wt 部分（见下方说明）	AMBER manual
ntb=1	1	恒体积周期性边界（NVT 系综）	[SI p.S2]
ntp=0	0	无压力耦合（NVT）	[SI p.S2]
restraint_wt=210.0	210.0	位置约束：210 kcal/mol/A^2	[SI p.S2]
ioutfm=1	1	以 NetCDF 二进制格式写出轨迹（更紧凑、更快）。NetCDF 是一种压缩的二进制文件格式，你不能用文本编辑器打开它，但分析工具（cpptraj、MDAnalysis）可以直接读取。	[operational default]
&wt TYPE='TEMP0'	—	在 ISTEP1 到 ISTEP2 之间线性升温，从 VALUE1 到 VALUE2	AMBER manual

什么是 nmropt=1? 它启用后面的 &wt 部分。&wt 控制参数随时间的变化——这里用来在 ISTEP1 到 ISTEP2 步之间将温度从 VALUE1 线性升高到 VALUE2。

为什么时间步长不同? 加热阶段用 dt=0.001（1 fs）是因为强约束产生尖锐的力，需要小步长来安全处理。产出阶段用 dt=0.002（2 fs）是因为 SHAKE 约束（ntc=2）冻结了氢键长度，允许更大步长。

什么是 SHAKE? SHAKE 算法把涉及氢原子的键长固定在平衡值。这样我们可以用更大的时间步长（2 fs 而不是 1 fs），让模拟速度快一倍，同时不损失精度。

什么是恒温器（Thermostat）? 恒温器通过增加或减少动能来控制系统温度。Langevin 恒温器（ntt=3）模拟与隐形热浴的随机碰撞——就像蛋白质被恒温环境包围着。

```
cat > $WORK/runs/pftrps_ain_md/heat2.in << 'EOF'
PfTrpS(Ain) heating step 2; JACS 2019 SI Heating (50 ps NVT, 50->100 K, 165 kcal/mol/A
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=100.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=165.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=50.0, VALUE2=100.0,
/
&wt TYPE='END' /
EOF
```

heat2.in (50 -> 100 K)

与 heat1 的关键区别：irest=1 和 ntx=5 一告诉 AMBER 从上一步续跑（同时读取坐标和速度）。

```
cat > $WORK/runs/pftrps_ain_md/heat3.in << 'EOF'
PfTrpS(Ain) heating step 3; JACS 2019 SI Heating (50 ps NVT, 100->150 K, 125 kcal/mol/
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=150.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=125.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=100.0, VALUE2=150.0,
/
&wt TYPE='END' /
EOF
```

heat3.in (100 -> 150 K)

```
cat > $WORK/runs/pftrps_ain_md/heat4.in << 'EOF'
PfTrpS(Ain) heating step 4; JACS 2019 SI Heating (50 ps NVT, 150->200 K, 85 kcal/mol/A
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
```

```
  gamma_ln=1.0,
  temp0=200.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=85.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=150.0, VALUE2=200.0,
/
&wt TYPE='END' /
EOF
```

heat4.in (150 -> 200 K)

```
cat > $WORK/runs/pftrps_ain_md/heat5.in << 'EOF'
PfTrpS(Ain) heating step 5; JACS 2019 SI Heating (50 ps NVT, 200->250 K, 45 kcal/mol/A
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=250.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=45.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
```

```
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=200.0, VALUE2=250.0,
/
&wt TYPE='END' /
EOF
```

heat5.in (200 -> 250 K)

```
cat > $WORK/runs/pftrps_ain_md/heat6.in << 'EOF'
PfTrpS(Ain) heating step 6; JACS 2019 SI Heating (50 ps NVT, 250->300 K, 10 kcal/mol/A
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=300.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=250.0, VALUE2=300.0,
/
&wt TYPE='END' /
EOF
```

heat6.in (250 -> 300 K)

heat7.in (300 -> 350 K)

注意：SI 为 7 个步骤列出了 6 个约束值。heat6 和 heat7 都使用 10.0 kcal/mol/A^2。[operational default]

```
cat > $WORK/runs/pftrps_ain_md/heat7.in << 'EOF'
PfTrpS(Ain) heating step 7; JACS 2019 SI Heating (50 ps NVT, 300->350 K, 10 kcal/mol/A
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=300.0, VALUE2=350.0,
/
&wt TYPE='END' /
EOF
```

升温步骤总结

步骤	温度	约束力常数 (kcal/mol/A^2)	来源
heat1	0 -> 50 K	210.0	[SI p.S2]
heat2	50 -> 100 K	165.0	[SI p.S2]
heat3	100 -> 150 K	125.0	[SI p.S2]
heat4	150 -> 200 K	85.0	[SI p.S2]
heat5	200 -> 250 K	45.0	[SI p.S2]
heat6	250 -> 300 K	10.0	[SI p.S2]
heat7	300 -> 350 K	10.0	[operational default]

Step 4.3: 创建平衡输入文件

> **做什么**：在 350 K、1 atm 下进行 2 ns NPT 平衡，不施加位置约束。
>
> **为什么加热后要做 NPT?** 加热阶段（NVT）盒子体积固定不变，所以密度可能不对。NPT 让盒子膨胀或收缩，达到正确的密度（水溶液蛋白约 1.0 g/cm3，1 atm）。[SI p.S3]

```
cat > $WORK/runs/pftrps_ain_md/equil.in << 'EOF'
PfTrpS(Ain) equilibration; JACS 2019 SI Equilibration (2 ns NPT, 350 K, 1 atm, no rest
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=1000000,
  dt=0.002,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  ntb=2,
  ntp=1,
  barostat=1,
  pres0=1.0,
  taup=1.0,
  cut=8.0,
  ntr=0,
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
EOF
```

参数	值	含义	来源
nstlim=1000000	1000000	1,000,000 步 x 0.002 ps = 2 ns	[SI p.S3]
dt=0.002	0.002	2 fs 时间步长（配合 SHAKE 使用的标准值）	[SI p.S3]
ntb=2	2	恒压周期性边界（NPT 系综，见下方说明）	[SI p.S3]
ntp=1	1	各向同性压力耦合 (isotropic pressure coupling)	[SI p.S3]
barostat=1	1	Berendsen 恒压器 (barostat，见下方说明)	[operational default]

参数	值	含义	来源
pres0=1.0	1.0	目标压力：1 atm	[operational default]
taup=1.0	1.0	压力弛豫时间：1 ps	[operational default]
ntr=0	0	无位置约束	[SI p.S3]

ntb=1 vs ntb=2：ntb=1 = 恒定体积（NVT，用于加热和产出）。ntb=2 = 恒定压力（NPT，用于平衡化以达到正确密度）。不同模拟阶段需要不同的系综。

什么是恒压器（Barostat）？ 恒压器通过调整模拟盒子体积来控制压力。压力太高时盒子膨胀，太低时收缩。在 NPT 平衡化阶段使用。

Step 4.4: 创建前处理流程的 Slurm 脚本

做什么：一个单独的 Slurm 任务，在 GPU 节点上按顺序运行 min1 -> min2 -> heat1-7 -> equil。

为什么：每个步骤都依赖上一步的 restart 文件。将它们作为一个任务运行，避免步骤之间的排队等待时间。

```
cat > $WORK/runs/pftrps_ain_md/run_md_pipeline.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_ain_prep
#SBATCH --partition=gpu
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=24:00:00
#SBATCH --output=$WORK/logs/pftrps_ain_prep_%j.out

# ===== Environment =====
module load amber/24p3

RUNDIR="$WORK/runs/pftrps_ain_md"
PARM="$WORK/systems/pftrps_ain/pftrps_ain.parm7"
INPCRD="$WORK/systems/pftrps_ain/pftrps_ain.inpcrd"

cd ${RUNDIR}

echo "========== PREP PIPELINE START: $(date) =========="

# ===== Minimization Stage 1: Restrained =====
echo "--- min1: restrained minimization ---"
pmemd.cuda -O \
  -i min1.in \
```

```
  -o min1.out \
  -p ${PARM} \
  -c ${INPCRD} \
  -r min1.rst7 \
  -ref ${INPCRD}
echo "min1 done: $(date)"

# ===== Minimization Stage 2: Unrestrained =====
echo "--- min2: unrestrained minimization ---"
pmemd.cuda -O \
  -i min2.in \
  -o min2.out \
  -p ${PARM} \
  -c min1.rst7 \
  -r min2.rst7
echo "min2 done: $(date)"

# ===== Heating Steps 1-7 =====
PREV_RST="min2.rst7"
for i in 1 2 3 4 5 6 7; do
  echo "--- heat${i} ---"
  pmemd.cuda -O \
    -i heat${i}.in \
    -o heat${i}.out \
    -p ${PARM} \
    -c ${PREV_RST} \
    -r heat${i}.rst7 \
    -x heat${i}.nc \
    -ref ${PREV_RST}
  PREV_RST="heat${i}.rst7"
  echo "heat${i} done: $(date)"
done

# ===== Equilibration (2 ns NPT) =====
echo "--- equil: 2 ns NPT equilibration ---"
pmemd.cuda -O \
  -i equil.in \
  -o equil.out \
  -p ${PARM} \
  -c heat7.rst7 \
  -r equil.rst7 \
  -x equil.nc
echo "equil done: $(date)"

echo "========== PREP PIPELINE COMPLETE: $(date) =========="
EOF
```

Step 4.5: 提交前处理流程

```
cd $WORK/runs/pftrps_ain_md
sbatch run_md_pipeline.sh
```

验证（任务完成后）：

```
# 检查所有输出文件是否存在：
ls -lh $WORK/runs/pftrps_ain_md/{min1,min2}.out
ls -lh $WORK/runs/pftrps_ain_md/heat{1,2,3,4,5,6,7}.out
ls -lh $WORK/runs/pftrps_ain_md/equil.out

# 检查 restart 文件：
ls -lh $WORK/runs/pftrps_ain_md/\
{min1,min2,heat1,heat2,heat3,heat4,heat5,heat6,heat7,equil}.rst7

# 检查最小化是否收敛（能量应下降）：
grep "NSTEP       ENERGY" $WORK/runs/pftrps_ain_md/min1.out | tail -3

# 检查 heat7 之后的最终温度是否为 ~350 K：
grep "TEMP(K)" $WORK/runs/pftrps_ain_md/heat7.out | tail -1
# Expected: ~350 K（在涨落范围内）

# 检查平衡后的密度是否为 ~1.0 g/cm^3：
grep "Density" $WORK/runs/pftrps_ain_md/equil.out | tail -5
# Expected: ~1.0 g/cm^3

# 检查错误：
grep -l "ERROR\|FATAL\|NaN" $WORK/runs/pftrps_ain_md/*.out
# Expected:（无输出 = 正常）
```

Phase 5: 经典 MD —500 ns 产出运行（~3 天）

> **目标**：在 350 K 下运行 500 ns 无偏置 NVT 分子动力学，采样 Ain 构象系综（conformational ensemble）。
>
> **为什么**：常规 MD 轨迹有两个用途：(1) 为 MetaDynamics 提供平衡后的起始结构，(2) 比较有无增强采样偏置时的构象空间采样情况。[SI p.S3]

Step 5.1: 创建产出运行输入文件

```
cat > $WORK/runs/pftrps_ain_md/prod.in << 'EOF'
PfTrpS(Ain) production; JACS 2019 SI Production MD (500 ns NVT, 350 K, PME, 8 A cutoff
&cntrl
  imin=0,
  irest=1,
```

```
  ntx=5,
  nstlim=250000000,
  dt=0.002,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=0,
  ntpr=5000,
  ntwx=5000,
  ntwe=500,
  ntwr=500000,
  ioutfm=1,
/
EOF
```

参数	值	含义	来源
`nstlim=250000000`	250,000,000	250M 步 x 0.002 ps = 500 ns	[SI p.S3]
`ntb=1`	1	NVT（恒体积）用于产出运行	[SI p.S3]
`ntwe=500`	500	每 500 步写一次能量（= 1 ps，精细的能量监控）	[operational default]
`ntwr=500000`	500000	每 500K 步写一次 restart（= 1 ns，用于崩溃恢复）	[operational default]
`ntwx=5000`	5000	每 5000 步写一帧轨迹（= 10 ps）	[operational default]

> **轨迹大小估算**：500 ns / 10 ps = 50,000 帧。~39,000 个原子时，每个 NetCDF 帧约 0.5 MB。总计约 25 GB。

Step 5.2: 创建产出运行 Slurm 脚本

```
cat > $WORK/runs/pftrps_ain_md/submit_production.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_ain_prod
#SBATCH --partition=gpu
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
```

```
#SBATCH --mem=16G
#SBATCH --time=72:00:00
#SBATCH --output=$WORK/logs/pftrps_ain_prod_%j.out

module load amber/24p3

RUNDIR="$WORK/runs/pftrps_ain_md"
PARM="$WORK/systems/pftrps_ain/pftrps_ain.parm7"

cd ${RUNDIR}

echo "========== PRODUCTION START: $(date) =========="

pmemd.cuda -O \
  -i prod.in \
  -o prod.out \
  -p ${PARM} \
  -c equil.rst7 \
  -r prod.rst7 \
  -x prod.nc

echo "========== PRODUCTION COMPLETE: $(date) =========="
EOF
```

Step 5.3: 提交产出运行

```
sbatch $WORK/runs/pftrps_ain_md/submit_production.sh
```

验证（运行中和完成后）：

```
# 监控任务状态：
squeue -u $USER

# 检查进度（已写入的帧数）：
# 轨迹文件会随时间增长
ls -lh $WORK/runs/pftrps_ain_md/prod.nc

# 完成后，检查是否正常终止：
tail -20 $WORK/runs/pftrps_ain_md/prod.out | grep "Total wall time"
# 应显示总运行时间

# 验证轨迹帧数：
module load amber/24p3
cpptraj -p $WORK/systems/pftrps_ain/pftrps_ain.parm7 \
  -y $WORK/runs/pftrps_ain_md/prod.nc \
  -tl
# Expected: 50,000 帧
```

```
# 快速 RMSD 检查（平衡后应趋于平稳）：
cpptraj -p $WORK/systems/pftrps_ain/pftrps_ain.parm7 << CPPTRAJ_EOF
trajin $WORK/runs/pftrps_ain_md/prod.nc
rms first @CA out $WORK/analysis/prod_rmsd.dat
go
CPPTRAJ_EOF
head -20 $WORK/analysis/prod_rmsd.dat
```

Phase 6: AMBER 到 GROMACS 格式转换 [DRAFT —not yet verified]（~30 分钟）

目标：将平衡后的 AMBER 体系（parm7 + restart）转换为 GROMACS 格式（gro + top），用于 PLUMED MetaDynamics。

为什么：JACS 2019 原文使用 GROMACS 5.1.2 + PLUMED 2 进行 MetaDynamics。我们严格复刻此选择：常规 MD 在 AMBER 中运行（等价的 ff14SB），MetaDynamics 在 GROMACS + PLUMED 中运行。[SI p.S3]

Step 6.1: 使用 ParmEd 转换拓扑

做什么：ParmEd 可以读取 AMBER parm7/inpcrd 并写出 GROMACS top/gro 文件。

```
cat > $WORK/scripts/convert_amber_to_gromacs.py << 'EOF'
#!/usr/bin/env python3
"""
Convert AMBER topology/coordinates to GROMACS format using ParmEd.
[DRAFT -- not yet verified]

Input:  pftrps_ain.parm7 + equil.rst7 (or prod.rst7)
Output: pftrps_ain.top + pftrps_ain.gro
"""
import parmed as pmd

# Load AMBER files
parm = pmd.load_file(
    "$WORK/systems/pftrps_ain/pftrps_ain.parm7",
    "$WORK/runs/pftrps_ain_md/equil.rst7"
)

# Save as GROMACS format
parm.save("$WORK/systems/pftrps_ain/pftrps_ain.top", overwrite=True)
parm.save("$WORK/systems/pftrps_ain/pftrps_ain.gro", overwrite=True)

print("Conversion complete.")
print(f"Atoms: {len(parm.atoms)}")
print(f"Residues: {len(parm.residues)}")
EOF
```

```
module load anaconda/2024.02 && conda activate trpb-md
python3 $WORK/scripts/convert_amber_to_gromacs.py
```

验证:

```
ls -lh $WORK/systems/pftrps_ain/pftrps_ain.{top,gro}
# 两个文件都应存在

# 检查 .gro 中的原子数是否与 AMBER 一致:
head -2 $WORK/systems/pftrps_ain/pftrps_ain.gro
# 第二行应显示总原子数 (如 39268)

# 检查 .gro 中的盒子向量 (最后一行):
tail -1 $WORK/systems/pftrps_ain/pftrps_ain.gro
# 应显示以 nm 为单位的盒子尺寸 (AMBER 的 Angstroms 除以 10)
```

Step 6.2: 生成 GROMACS .tpr 文件

做什么: 创建 GROMACS 二进制运行输入文件 (.tpr), 将拓扑、坐标和模拟参数合并。

```
cat > $WORK/runs/pftrps_ain_metad/em.mdp << 'EOF'
; Energy minimization MDP for GROMACS (post-conversion sanity check)
; [DRAFT -- not yet verified]
integrator = steep
emtol      = 1000.0
emstep     = 0.01
nsteps     = 5000
nstlist    = 1
cutoff-scheme = Verlet
coulombtype = PME
rcoulomb    = 0.8
rvdw        = 0.8
pbc         = xyz
EOF

module load anaconda/2024.02 && conda activate trpb-md

gmx grompp \
  -f $WORK/runs/pftrps_ain_metad/em.mdp \
  -c $WORK/systems/pftrps_ain/pftrps_ain.gro \
  -p $WORK/systems/pftrps_ain/pftrps_ain.top \
  -o $WORK/runs/pftrps_ain_metad/em.tpr \
  -maxwarn 5
```

验证:

```
ls -lh $WORK/runs/pftrps_ain_metad/em.tpr
# 应存在, ~2-5 MB
```

```
# 快速能量最小化以测试拓扑：
gmx mdrun -v -deffnm $WORK/runs/pftrps_ain_metad/em -ntomp 4
# 应无错误地完成
```

> **FP-004 警告**：格式转换后，所有原子索引都会改变。PLUMED 输入文件必须使用 GROMACS 原子编号，而不是原始 PDB 或 AMBER 编号。

Phase 7: Path CV 构建 [DRAFT —not yet verified]（~10 分钟）

> **目标**：构建 PATHMSD 集体变量（collective variable）的参考路径，由 Open (1WDW) 和 Closed (3CEP) 构象之间的 15 个中间帧组成。
>
> **Path CV 测量什么**：两个量：- **s(R)**：沿 O -> C 构象路径的进展（范围从 1 到 15）- **z(R)**：与最近参考帧的距离（衡量当前结构偏离参考路径的程度）
>
> **来源**：[SI p.S3] —15 帧，残基 97-184 + 282-305 的 Calpha 原子

Step 7.1: 对齐 open 和 closed 结构

> **做什么**：将 1WDW (open) 和 3CEP (closed) 按 COMM 域 Calpha 原子叠合 (superimpose)，然后线性插值（linearly interpolate）15 帧。
>
> **为什么**：path CV 需要一系列参考结构来平滑连接两个端点。笛卡尔空间（Cartesian space）中的线性插值是最简单的方法。[SI p.S3]

```
cat > $WORK/scripts/generate_path_cv.py << 'EOF'
#!/usr/bin/env python3
"""
Generate 15-frame reference path from 1WDW (open) to 3CEP (closed).
[DRAFT -- not yet verified]

Atoms used: Calpha of residues 97-184 and 282-305 [SI p.S3]
Interpolation: Linear in Cartesian coordinates [SI p.S3]

Output: path.pdb (multi-model PDB for PLUMED PATHMSD)
"""
import numpy as np
import warnings

# [DRAFT] This script needs:
# 1. MDAnalysis or BioPython to read PDBs
# 2. Sequence alignment to map 1WDW residues to 3CEP residues
#    (3CEP is S. typhimurium TrpS, different species)
# 3. Calpha extraction for residues 97-184 and 282-305
# 4. Rigid-body superposition (Kabsch algorithm)
# 5. Linear interpolation of 15 frames
# 6. MSD calculation between adjacent frames
```

```
# 7. Lambda calculation: lambda = 2.3 / mean_MSD [SI p.S3]

print("[DRAFT] Path CV generation script -- not yet implemented")
print("Expected output: path.pdb with 15 MODELs")
print("Expected lambda: ~0.034 A^-2 (our calculation) vs ~0.029 (SI)")
EOF
```

Step 7.2: Lambda 计算

关键公式：`lambda = 2.3 / <MSD>`，其中 <MSD> 是相邻路径帧之间的均方位移（mean squared displacement），取所有 14 个区间的平均值。`[SI p.S3]`

2.3 从哪来？ 2.3 是 PLUMED 的 PATHMSD 实现中使用的标准缩放因子，来自路径 CV 的原始论文（Branduardi et al., 2007）。lambda = 2.3 / MSD 确保路径 CV 对相邻帧之间的结构变化有平滑响应。

数值	SI 报告值	我们的计算值	来源
Mean MSD	80 A^2	67.826 A^2	`[SI p.S3]`
Lambda	0.029 A^-2	0.033910 A^-2	由 MSD 推导

偏差说明：MSD 的 15% 差异可能来源于不同的叠合方法。SI 未指明计算 MSD 前使用哪些原子进行对齐。我们当前的值 (0.033910) 使用了所有选定 Calpha 原子的总 MSD。

FP-015 guard：`calculate_msd()` 函数必须返回 MSD（单位：A^2），**而不是** RMSD（单位：A）。之前的一个 bug 对 MSD 做了 `np.sqrt()`，产生了 RMSD，导致 lambda = 3.798 而非 0.034。

Phase 8: Well-Tempered MetaDynamics（~2–3 天）

目标：使用 GROMACS + PLUMED 运行带 path CVs 的 well-tempered MetaDynamics，采样 COMM 域完整的 Open-to-Closed 构象景观。

什么是 MetaDynamics（元动力学）？ MetaDynamics 在已访问过的状态上添加小的能量"凸起"（高斯山丘），推动系统探索新的构象。随时间推移，累积的山丘填满能量盆地，山丘总量的负值就是自由能面。

什么是 Path CV（路径集合变量）？ Path CV 衡量当前蛋白结构与一系列参考结构的相似程度。s(R) 告诉你"你在路径上的哪个位置"（1= 开放态，15= 关闭态）。z(R) 告诉你"你离路径有多远"。

什么是 Well-tempered MetaDynamics？ 在标准 MetaDynamics 中，山丘高度始终不变。Well-tempered 版本中，随着偏置增长，山丘高度逐渐降低，防止过度填充，让模拟收敛到正确的自由能面。

方案：先进行单 walker 探索，然后进行 10-walker 产出运行。`[SI pp.S3–S4]`

关键：调试中发现的兼容性问题（2026-04-04 验证）

在 GROMACS 2026 上运行 PLUMED path CV 时发现了三个兼容性问题。以下记录并反映在本节所有代码块中。

1. **FUNCPATHMSD 替代 PATHMSD**：经典的 PATHMSD 动作在内部读取一个多 MODEL 的 PDB 参考文件。但 GROMACS 2026 的 `mdrun` 接口无法正确解析原子序号不连续的 PDB 文件（例如只选 Calpha 的情况）。解决方案是 FUNCPATHMSD：定义 15 个独立的 RMSD 动作（每个参考帧一个），然后传给 FUNCPATHMSD，它使用**数学上完全相同**的路径 CV 公式来计算 s(R) 和 z(R)。
2. **LAMBDA 单位转换（Angstrom 到 nm）**：LAMBDA 的计算公式是 1 / (N−1) / <MSD>，其中 MSD 的单位是长度的平方。原始论文（AMBER/Angstrom）中 LAMBDA 单位是 A^-2。GROMACS 内部使用纳米，因此 **LAMBDA 必须乘以 100**（因为 1 nm^2 = 100 A^2）。我们计算的值：0.033910 A^-2 –> **3.3910 nm^-2**。
3. **不能使用反斜杠续行**：当 PLUMED 输入通过 `gmx mdrun -plumed` 传入时，反斜杠（\）跨行续写不能被可靠解析。单个 PLUMED 动作的所有参数必须写在**同一行**。以下代码块均使用单行格式。
4. **必须源码编译 PLUMED**：conda-forge 的 PLUMED 包自带的 `libplumedKernel.so` 不完整，运行时会崩溃。必须从源码编译（`./configure --enable-modules=all`），并通过 PLUMED_KERNEL 导出编译后的库路径。

Step 8.1: 创建 GROMACS MDP 文件用于 MetaDynamics

```
# 目录名称可以随意取——"pftrps_ain_metad" 只是约定，不是硬性要求
mkdir -p $WORK/runs/pftrps_ain_metad

cat > $WORK/runs/pftrps_ain_metad/metad.mdp << 'EOF'
; GROMACS MDP for well-tempered MetaDynamics
;
; Reference: JACS 2019 SI p.S3
; Engine: GROMACS 5.1.2 (original) -> GROMACS 2026.0 (ours)

integrator   = md              ; Leap-frog integrator
dt           = 0.002           ; 2 fs timestep [SI p.S3]
nsteps       = 25000000        ; 25M steps = 50 ns per walker [SI p.S4]

; Output control
nstxout-compressed = 5000      ; Write compressed trajectory every 10 ps
nstlog       = 5000            ; Write log every 10 ps
nstenergy    = 5000            ; Write energy every 10 ps

; Neighbor searching
nstlist      = 10              ; Update neighbor list every 10 steps
cutoff-scheme = Verlet          ; Verlet cutoff scheme (GROMACS default)

; Electrostatics
coulombtype  = PME             ; Particle Mesh Ewald [SI p.S3]
rcoulomb     = 0.8             ; 8 A cutoff in nm [SI p.S3]

; Van der Waals
rvdw         = 0.8             ; 8 A cutoff in nm [SI p.S3]
```

```
; Temperature coupling
tcoupl       = v-rescale       ; Velocity rescaling thermostat[^4]
tc-grps      = System          ; Single coupling group
tau_t        = 0.1             ; Coupling time constant (ps)
ref_t        = 350             ; 350 K [SI p.S3]

; Pressure coupling
pcoupl       = no              ; NVT (no pressure coupling) [SI p.S3]

; Bond constraints
constraints  = h-bonds         ; Constrain H-bonds (SHAKE) [SI p.S3]
constraint_algorithm = LINCS   ; LINCS algorithm (GROMACS default)

; Continuation from AMBER equilibration
continuation = yes             ; Do not generate velocities
gen_vel      = no              ; Read from .gro / .tpr
EOF
```

Step 8.2: 创建 PLUMED 输入文件用于单 walker MetaDynamics

MetaDynamics 参数说明：

参数	值	含义	来源
`HEIGHT=0.628`	0.628 kJ/mol	高斯山丘高度（hill height）。SI 报告 0.15 kcal/mol；0.15 x 4.184 = 0.628 kJ/mol（PLUMED 使用 kJ/mol）。	`[SI p.S3]`
`PACE=1000`	每 1000 步	每 1000 个 MD 步沉积一个新山丘。在 dt=0.002 ps 下，即每 2 ps。	`[SI p.S3]`
`BIASFACTOR=10`	10	Well-tempered MetaD 偏置因子（bias factor）。CV 的有效温度为 T_eff = T * (1 + 1/gamma) = 350 * 1.1 = 385 K。值越高探索越广，但自由能精度越低。	`[SI p.S3]`
`TEMP=350`	350 K	体系温度（必须与 MD 恒温器匹配）。	`[SI p.S3]`
`SIGMA=0.2,0.1`	0.2 (s), 0.1 (z)	分别对应 s 和 z 两个 CV 的高斯宽度。	`[SI p.S3]`

参数	值	含义	来源
LAMBDA=3.3910	3.3910 nm^-2	FUNCPATHMSD 平滑参数。控制每个参考帧"吸引"结构的锐度。**从 0.033910 A^-2 x 100 = 3.3910 nm^-2 转换，适配 GROMACS 的 nm 单位。**	由我们的 MSD 计算；SI 报告 0.029 A^-2

LAMBDA 单位转换（关键）：平滑参数 LAMBDA 的单位是长度的反平方。我们计算的值是 0.033910 A^-2。GROMACS 内部使用纳米，因此传给 PLUMED 的所有距离都是 nm。因为 1 nm = 10 A，所以 1 nm^-2 = 0.01 A^-2，因此**乘以 100**：0.033910 A^-2 x 100 = **3.3910 nm^-2**。如果使用未转换的值（0.033910），所有帧看起来距离几乎相等，path CV 将无法区分不同构象。

单位转换：论文中山丘高度写的是 0.15 kcal/mol，但 PLUMED 内部使用 kJ/mol。转换：0.15 kcal/mol x 4.184 = 0.628 kJ/mol。PLUMED 输入文件里永远用 kJ/mol。

PACE 与时间的关系：PACE=1000 表示每 1000 个 MD 步沉积一个高斯山丘。dt=0.002 ps 时，1000 x 0.002 = 2.0 ps 沉积一次。这与 SI 的规定一致。

什么是 BIASFACTOR? 它控制 well-tempered MetaDynamics 中山丘高度的衰减速度。BIASFACTOR=10 表示集合变量的有效温度约为 T x (1 + 1/gamma) = 350 x 1.1 = 385 K。值越大探索越激进，但收敛越慢。

为什么用 FUNCPATHMSD 而不是 PATHMSD? 经典 PATHMSD 动作在内部读取一个多 MODEL 的 PDB 文件。当 GROMACS 2026 `mdrun` 向 PLUMED 传递坐标时，原子序号可能不连续（例如只选 Calpha 的情况会跳过大部分原子）。PATHMSD 的 PDB 解析器无法正确处理这种情况，导致静默的索引错位或崩溃。解决方案：定义 15 个独立的 RMSD 动作（每个参考帧一个 PDB），然后输入 FUNCPATHMSD。这使用**数学上完全相同**的指数公式计算 s(R) 和 z(R)，但完全避免了 PDB 解析问题。

不能使用反斜杠续行：当 PLUMED 输入通过 `gmx mdrun -plumed` 接口读取时，反斜杠跨行续写（\）不能被可靠解析。单个 PLUMED 动作的所有关键字必须在**同一行**。以下代码遵循此约定。

```
# 准备逐帧参考 PDB 目录
# 每个 frame_XX.pdb 包含 COMM 域残基 97-184, 282-305 的 Calpha 原子
# 坐标单位为纳米（GROMACS 约定）
mkdir -p $WORK/runs/pftrps_ain_metad/frames
#  （将 15 个参考 PDB 复制到此处：frame_01.pdb ... frame_15.pdb）

cat > $WORK/runs/pftrps_ain_metad/plumed_trpb_metad.dat << 'EOF'
# PLUMED 2.9 single-walker WT-MetaD for PfTrpS(Ain)
# Verified 2026-04-04
#
# Reference: JACS 2019 SI pp.S3-S4
# CVs: path collective variables s(R) and z(R) via FUNCPATHMSD
#   s(R) = progress along O->C path (ranges 1-15)
```

```
#   z(R) = deviation from reference path
#
# FP-004 guard: atom indices in frame PDBs must match GROMACS numbering,
# not PDB or AMBER numbering.
#
# NOTE: All PLUMED actions on single lines (no backslash continuation)
# because gmx mdrun -plumed does not reliably parse line continuations.

# 15 individual RMSD references (one per frame along O->C path)
r1: RMSD REFERENCE=frames/frame_01.pdb TYPE=OPTIMAL
r2: RMSD REFERENCE=frames/frame_02.pdb TYPE=OPTIMAL
r3: RMSD REFERENCE=frames/frame_03.pdb TYPE=OPTIMAL
r4: RMSD REFERENCE=frames/frame_04.pdb TYPE=OPTIMAL
r5: RMSD REFERENCE=frames/frame_05.pdb TYPE=OPTIMAL
r6: RMSD REFERENCE=frames/frame_06.pdb TYPE=OPTIMAL
r7: RMSD REFERENCE=frames/frame_07.pdb TYPE=OPTIMAL
r8: RMSD REFERENCE=frames/frame_08.pdb TYPE=OPTIMAL
r9: RMSD REFERENCE=frames/frame_09.pdb TYPE=OPTIMAL
r10: RMSD REFERENCE=frames/frame_10.pdb TYPE=OPTIMAL
r11: RMSD REFERENCE=frames/frame_11.pdb TYPE=OPTIMAL
r12: RMSD REFERENCE=frames/frame_12.pdb TYPE=OPTIMAL
r13: RMSD REFERENCE=frames/frame_13.pdb TYPE=OPTIMAL
r14: RMSD REFERENCE=frames/frame_14.pdb TYPE=OPTIMAL
r15: RMSD REFERENCE=frames/frame_15.pdb TYPE=OPTIMAL

# Path CV via FUNCPATHMSD -- mathematically identical to PATHMSD
# LAMBDA=3.3910 nm^-2 (= 0.033910 A^-2 x 100, unit conversion for GROMACS)
path: FUNCPATHMSD ARG=r1,r2,r3,r4,r5,r6,r7,r8,r9,r10,r11,r12,r13,r14,r15 LAMBDA=3.3910

# Well-tempered MetaDynamics biasing s(R) and z(R) simultaneously
metad: METAD ARG=path.s,path.z SIGMA=0.2,0.1 HEIGHT=0.628 PACE=1000 BIASFACTOR=10 TEMP

# Print CVs and bias for analysis
PRINT ARG=path.s,path.z,metad.bias FILE=COLVAR STRIDE=500
EOF
```

Step 8.3: 创建 MetaDynamics 的 Slurm 脚本

> **源码编译 PLUMED**：conda-forge 的 PLUMED 包自带的 `libplumedKernel.so` 不完整，GROMACS 加载时会崩溃。必须从源码编译 PLUMED（`./configure --enable-modules=all && make -j8 && make install`）并导出内核路径。以下 Slurm 脚本假设 PLUMED 安装在 `$WORK/software/plumed2`。

```
cat > $WORK/runs/pftrps_ain_metad/submit_metad.slurm << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_metad
#SBATCH --partition=gpu
```

```
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=72:00:00
#SBATCH --output=$WORK/logs/pftrps_metad_%j.out

# Load environment
module load anaconda/2024.02
eval "$(conda shell.bash hook)"
conda activate trpb-md

# Use source-compiled PLUMED (conda-forge libplumedKernel.so is incomplete)
export PLUMED_KERNEL=$WORK/software/plumed2/lib/libplumedKernel.so

# FP-006 guard: set OMP_NUM_THREADS to match cpus-per-task
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}

RUNDIR="$WORK/runs/pftrps_ain_metad"
cd ${RUNDIR}

echo "========== METAD START: $(date) ==========="

# Generate .tpr
gmx grompp \
  -f metad.mdp \
  -c $WORK/systems/pftrps_ain/pftrps_ain.gro \
  -p $WORK/systems/pftrps_ain/pftrps_ain.top \
  -o metad.tpr \
  -maxwarn 5

# Run with PLUMED
gmx mdrun -v \
  -deffnm metad \
  -plumed plumed_trpb_metad.dat \
  -ntomp ${SLURM_CPUS_PER_TASK}

echo "========== METAD COMPLETE: $(date) ==========="
EOF
```

Step 8.4: 多 walker 方案（10 个 walker）

> **做什么**：运行 10 个独立的 MetaDynamics walker，共享 HILLS 文件。每个 walker 在共享目录中沉积山丘，所有 walker 都从中读取。这加速了收敛，因为每个 walker 都受益于所有其他 walker 累积的偏置势。[SI p.S4]

```
cat > $WORK/runs/pftrps_ain_metad/plumed_trpb_metad_multiwalker.dat << 'EOF'
# PLUMED 2.9 multi-walker WT-MetaD for PfTrpS(Ain)
# Verified 2026-04-04
#
# 10 walkers, 50-100 ns each = 500-1000 ns total [SI p.S4]
# Walkers share HILLS via filesystem
#
# NOTE: All PLUMED actions on single lines (no backslash continuation)

r1: RMSD REFERENCE=frames/frame_01.pdb TYPE=OPTIMAL
r2: RMSD REFERENCE=frames/frame_02.pdb TYPE=OPTIMAL
r3: RMSD REFERENCE=frames/frame_03.pdb TYPE=OPTIMAL
r4: RMSD REFERENCE=frames/frame_04.pdb TYPE=OPTIMAL
r5: RMSD REFERENCE=frames/frame_05.pdb TYPE=OPTIMAL
r6: RMSD REFERENCE=frames/frame_06.pdb TYPE=OPTIMAL
r7: RMSD REFERENCE=frames/frame_07.pdb TYPE=OPTIMAL
r8: RMSD REFERENCE=frames/frame_08.pdb TYPE=OPTIMAL
r9: RMSD REFERENCE=frames/frame_09.pdb TYPE=OPTIMAL
r10: RMSD REFERENCE=frames/frame_10.pdb TYPE=OPTIMAL
r11: RMSD REFERENCE=frames/frame_11.pdb TYPE=OPTIMAL
r12: RMSD REFERENCE=frames/frame_12.pdb TYPE=OPTIMAL
r13: RMSD REFERENCE=frames/frame_13.pdb TYPE=OPTIMAL
r14: RMSD REFERENCE=frames/frame_14.pdb TYPE=OPTIMAL
r15: RMSD REFERENCE=frames/frame_15.pdb TYPE=OPTIMAL

path: FUNCPATHMSD ARG=r1,r2,r3,r4,r5,r6,r7,r8,r9,r10,r11,r12,r13,r14,r15 LAMBDA=3.3910

metad: METAD ARG=path.s,path.z SIGMA=0.2,0.1 HEIGHT=0.628 PACE=1000 BIASFACTOR=10 TEMP

PRINT ARG=path.s,path.z,metad.bias FILE=COLVAR STRIDE=500
EOF
```

PLUMED 多 walker 参数	值	含义	来源
`WALKERS_N=10`	10	walker 总数	[SI p.S4]
`WALKERS_ID=__WALKER_ID__`	0-9	当前 walker 的 ID（由启动脚本替换）	—
`WALKERS_DIR=../shared_bias`	—	所有 walker 读写 HILLS 的目录	—
`WALKERS_RSTRIDE=1000`	1000	每 1000 步检查其他 walker 的新山丘	[operational default]

Walker 起始结构：从初始单 walker MetaDynamics 运行中提取 10 个快照。这些快照应跨越 s(R) CV 的不同区域，以最大化探索效率。[SI p.S4]

验证（多 walker 运行期间）：

```
# 检查所有 10 个 walker 是否在运行：
squeue -u $USER | grep pftrps

# 检查 HILLS 文件是否正在写入：
ls -lh $WORK/runs/pftrps_ain_metad/shared_bias/HILLS*
# 应显示 10 个文件（HILLS.0 到 HILLS.9），都在增长

# 检查每个 walker 的 COLVAR：
wc -l $WORK/runs/pftrps_ain_metad/walker_*/COLVAR
# 每个 walker 应有行数持续累积
```

Phase 9: 自由能面重构 [DRAFT —not yet verified]（~1 小时）

目标：从累积的 HILLS 文件重构自由能面（free energy surface, FES），并识别 Open、Partially Closed 和 Closed 势阱（basins）。

Step 9.1: 合并所有 walker 的 HILLS 文件

```
cd $WORK/runs/pftrps_ain_metad/shared_bias

# Concatenate all HILLS files (remove duplicate headers)
cat HILLS.0 > HILLS_all
for i in 1 2 3 4 5 6 7 8 9; do
  grep -v "^#" HILLS.${i} >> HILLS_all
done

# Sort by time:
sort -n -k1 HILLS_all > HILLS_all_sorted
```

Step 9.2: 用 sum_hills 重构自由能面（FES）

做什么：PLUMED 的 `sum_hills` 工具读取 HILLS 文件，通过对所有沉积的高斯山丘进行 well-tempered 校正求和来重构自由能面。

关键参数：`--kt 0.695` = k_B * T = 0.001987 kcal/mol/K * 350 K = 0.695 kcal/mol。这是 well-tempered 重加权（reweighting）所需的。

```
module load anaconda/2024.02 && conda activate trpb-md

plumed sum_hills \
  --hills $WORK/runs/pftrps_ain_metad/shared_bias/HILLS_all_sorted \
  --outfile $WORK/analysis/fes.dat \
  --mintozero \
  --kt 0.695
```

sum_hills 标志	含义
`--hills`	输入 HILLS 文件
`--outfile`	输出 FES 网格
`--mintozero`	将 FES 移位，使全局最低点为 0 kcal/mol
`--kt`	k_B*T，用于 well-tempered 校正：350 K 时为 0.695 kcal/mol

Step 9.3: 识别势阱（basins）

状态定义来自 SI [SI p.S4]：

状态	s(R) 范围	描述
Open (O)	1–5	COMM 域打开，底物通道可通行
Partially Closed (PC)	5–10	中间态
Closed (C)	10–15	COMM 域关闭，催化活性构象

验证：

```
ls -lh $WORK/analysis/fes.dat
# 应存在，根据网格分辨率约 ~1-10 MB

# 检查 FES 是否有数据：
wc -l $WORK/analysis/fes.dat
# 应有大量行（网格点）

# 找到全局最低点：
sort -n -k3 $WORK/analysis/fes.dat | head -1
# 显示自由能最低的 (s, z, FES) 点

# 收敛性检查：仅使用 HILLS 的前半部分和后半部分分别重建 FES
# 如果两个 FES 曲线在 ~1 kcal/mol 内一致，则 MetaD 已收敛
```

Step 9.4: 收敛性评估

方法来自 SI [SI p.S4]：绘制 Open 和 Closed 势阱之间的自由能差 (Delta_G_OC) 随累积模拟时间的变化。当 Delta_G_OC 趋于平稳（波动 < 1 kcal/mol）时，MetaDynamics 已收敛。

```
# Blockwise FES reconstruction (split HILLS into time blocks):
for block in 10 20 30 40 50; do
  head -n ${block}000 $WORK/runs/pftrps_ain_metad/shared_bias/HILLS_all_sorted \
    > /tmp/hills_block_${block}ns

  plumed sum_hills \
    --hills /tmp/hills_block_${block}ns \
    --outfile $WORK/analysis/fes_block_${block}ns.dat \
    --mintozero \
```

```
    --kt 0.695
done
```

附录 A: 参数参考表

完整 AMBER .in 参数表

参数	使用的值	含义	来源
imin	1	运行能量最小化（非 MD）	AMBER manual
imin	0	运行分子动力学模拟	AMBER manual
ntx	1	仅从输入读取坐标（无速度）	AMBER manual
ntx	5	从 restart 文件读取坐标和速度	AMBER manual
irest	0	不续跑；从头开始新模拟	AMBER manual
irest	1	续跑：使用 restart 文件继续上次运行	AMBER manual
maxcyc	10000	尝试的最大最小化循环次数	[operational default]
ncyc	5000	在此步数后从最陡下降法切换到共轭梯度法。最陡下降法对初始大位移鲁棒；共轭梯度法在接近最低点时收敛更快。	[operational default]
nstlim	50000	MD 步数。50,000 x 0.001 ps = 50 ps（升温步骤）	[SI p.S2]
nstlim	1000000	MD 步数。1,000,000 x 0.002 ps = 2 ns（平衡）	[SI p.S3]
nstlim	250000000	MD 步数。250,000,000 x 0.002 ps = 500 ns（产出）	[SI p.S3]
dt	0.001	时间步长 (ps)，即 1 fs。用于带约束的升温阶段以保证稳定性。	[operational default]
dt	0.002	时间步长 (ps)，即 2 fs。需要对含 H 键施加 SHAKE 约束。	[SI p.S3]

参数	使用的值	含义	来源
ntc	2	SHAKE：约束所有涉及氢原子的键。允许使用 2 fs 时间步长。	[SI p.S3]
ntf	2	不计算 SHAKE 约束键的力。必须与 ntc=2 匹配。	[SI p.S3]
ntt	3	Langevin 恒温器：通过添加随机力和摩擦力维持温度。对生物分子比速度重缩放更鲁棒。	[SI p.S3]
gamma_ln	1.0	Langevin 碰撞频率 (ps^-1)。控制与热浴的耦合强度。低值 (1.0) 为弱耦合；高值 (5.0) 为强耦合。	[operational default]
tempi	0.0	速度分配的初始温度（仅在 heat1 中使用，从 0 K 开始）	AMBER manual
temp0	50.0–350.0	目标温度 (K)。随升温步骤变化；最终值 350 K，适用于 *P. furiosus*（嗜热菌）。	[SI p.S2–S3]
nmropt	1	启用 NMR 式约束。这里用于 &wt 部分的线性温度爬升。	AMBER manual
ntb	1	恒体积周期性边界条件（NVT 系综）。用于升温和产出。	[SI p.S2–S3]
ntb	2	恒压周期性边界条件 (NPT 系综)。用于平衡。	[SI p.S3]
ntp	0	无压力耦合（NVT）。	[SI p.S2]
ntp	1	各向同性压力耦合 (NPT)。盒子尺寸在所有方向上均匀调整。	[SI p.S3]
barostat	1	Berendsen 恒压器：弱耦合压力控制。适合平衡阶段；对产出运行不严格（不产生精确的 NPT 系综）。	[operational default]
pres0	1.0	目标压力 (bar)，约 1 atm。标准环境压力。	[operational default]
taup	1.0	压力弛豫时间 (ps)。控制盒子对压力偏差的响应速度。	[operational default]

参数	使用的值	含义	来源
cut	8.0	非键相互作用截断距离 (Angstroms)。超过此距离的相互作用由 PME（静电）处理或截断（范德华）。	[SI p.S3]
ntr	1	对 restraintmask 指定的原子施加位置约束。谐波势将受约束原子拉向参考位置。	[SI p.S2]
ntr	0	无位置约束。原子自由运动。	—
restraint_wt	500.0	最小化时的约束力常数：500 kcal/mol/A^2。非常强；原子几乎不偏离参考位置。	[SI p.S2]
restraint_wt	210.0–10.0	升温步骤的约束力常数。逐步降低以允许蛋白质松弛。	[SI p.S2]
restraintmask	'!@H= & !:WAT & !:Na+'	约束的原子选择：除氢原子、水分子和 Na+ 离子外的所有原子。即约束蛋白质和辅因子的重原子。	[SI p.S2]
ntpr	500 / 5000	每 N 步向输出文件打印能量信息。值越小监控越详细。	[operational default]
ntwx	5000	每 N 步写一帧轨迹。在 dt=0.002 时，5000 步 = 帧间隔 10 ps。	[operational default]
ntwe	500 / 5000	每 N 步写入能量数据。	[operational default]
ntwr	50000 / 500000	每 N 步写入 restart 文件。用于崩溃恢复。	[operational default]
ioutfm	1	使用 NetCDF 二进制格式输出轨迹。比 ASCII 更紧凑、更快。	[operational default]

PLUMED 参数表

参数	值	含义	来源
LAMBDA	0.033910	PATHMSD 平滑参数 (A^-2)。控制帧分配的锐度。公式：2.3 / mean_MSD。	计算所得；SI 报告 0.029
HEIGHT	0.628	山丘高度 (kJ/mol)。SI：0.15 kcal/mol x 4.184 = 0.628 kJ/mol。	[SI p.S3]
PACE	1000	每 1000 步沉积一个山丘 = 每 2 ps（dt=0.002 ps 时）。	[SI p.S3]
BIASFACTOR	10	Well-tempered 偏置因子 (gamma)。控制有效 CV 温度：T_eff = T*(1+1/gamma)。	[SI p.S3]
TEMP	350	体系温度 (K)。必须与 GROMACS 恒温器匹配。	[SI p.S3]
SIGMA	0.05	自适应模式下的初始高斯宽度。会被 ADAPTIVE 覆盖。	[UNVERIFIED]
ADAPTIVE	GEOM	使用几何自适应高斯宽度。根据 CV 空间的局部度规张量调整山丘形状。	[SI p.S3]
WALKERS_N	10	共享偏置势的并行 walker 数量。	[SI p.S4]
WALKERS_RSTRIDE	1000	每 1000 步读取共享的 HILLS。	[operational default]

GROMACS MDP 参数表

参数	值	含义	来源
integrator	md	Leap-frog 积分器 (GROMACS 默认 MD 积分器)	GROMACS manual
dt	0.002	时间步长：2 fs	[SI p.S3]
nsteps	25000000	25M 步 = 每个 walker 50 ns	[SI p.S4]
tcoupl	v-rescale	速度重缩放恒温器[5]。产生正确的正则系综 (canonical ensemble)。	[operational default]
ref_t	350	目标温度：350 K	[SI p.S3]
pcoupl	no	无压力耦合（NVT）	[SI p.S3]

[5]Bussi, G.; Donadio, D.; Parrinello, M. Canonical sampling through velocity rescaling. *J. Chem. Phys.* **2007**, *126*, 014101.

参数	值	含义	来源
`coulombtype`	PME	Particle Mesh Ewald 用于长程静电	`[SI p.S3]`
`rcoulomb`	0.8	静电截断：0.8 nm = 8 A	`[SI p.S3]`
`rvdw`	0.8	范德华截断：0.8 nm = 8 A	`[SI p.S3]`
`constraints`	h-bonds	约束涉及氢原子的键（等价于 SHAKE）	`[SI p.S3]`
`continuation`	yes	从 AMBER 平衡续跑（不重置时间）	—
`gen_vel`	no	从输入读取速度（不生成）	—

附录 B: 常见问题排查

已知的 Failure Patterns

这些是本项目中发生过的错误，记录在 `replication/validations/failure-patterns.md` 中：

ID	问题	解决方案
FP-003	PDB 残基名称错误（写了“PLP”而非“LLP”）	始终从实际 PDB 文件中提取残基名称，绝不凭记忆
FP-004	GROMACS 转换后 PLUMED 原子索引错误	始终从 GROMACS `.gro` 文件确定索引，而非原始 PDB 或 AMBER 编号
FP-006	Slurm 脚本中未设置 `OMP_NUM_THREADS`，GROMACS 崩溃	在所有 Slurm 脚本中添加 `export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}`
FP-007	Slurm 脚本中 conda 路径错误	在 Longleaf 上使用 `module load anaconda/2024.02 && conda activate trpb-md`
FP-009	AM1-BCC 对 PLP 失败 (SCF 发散)	使用 Gaussian QM 的 RESP 电荷；PLP 禁止使用 BCC
FP-010	antechamber 在没有氢原子时错误分配原子类型	在 antechamber 之前先运行 `reduce` 添加氢原子
FP-012	奇数电子数 -> Gaussian 中 singlet 不可能	提交前验证 `(Z_total - charge) % 2 == 0`
FP-013	`iop(6/50=1)` 导致 Gaussian 16 错误	不使用 `iop(6/50=1)`；改用 antechamber 的 `-fi gout`
FP-015	`calculate_msd()` 返回了 RMSD，lambda 偏差 130 倍	确保 MSD 函数返回平方量 (A^2)，而非平方根
FP-017	antechamber 忽略 `-rn` 标志，使用文件名作为残基名称	antechamber 之后用 `grep SUBSTRUCTURE` 验证 mol2 残基名称

常见问题

问：AMBER pmemd.cuda 崩溃并报“CUDA out of memory” > 减小体系大小或请求显存更大的 GPU。在 Longleaf 上，指定 `--gres=gpu:a100:1` 使用 A100 (80 GB)，而非默认 GPU。

问：tleap 报“Could not find unit AIN” > 确保在加载 PDB 之前已加载了 `.lib` 文件和 `.frcmod` 文件。`loadoff` 和 `loadamberparams` 命令必须在 `loadpdb` 之前。

问：Gaussian 16 SCF 不收敛 > 对于 PLP 类分子，尝试：(1) 使用 `SCF=(MaxCycle=512,XQC)` 扩展收敛。(2) 从更接近平衡的几何构型开始。(3) 检查电荷和多重度是否物理上合理。

问：GROMACS grompp 报“atom X not found in topology” > ParmEd 转换可能存在残基命名问题。检查 `.gro` 文件中的所有残基名称是否与 `.top` 文件中的条目匹配。

问：PLUMED PATHMSD 报“PDB file does not match” > 参考 PDB 的原子数和命名必须与运行中模拟中选定的原子完全匹配。使用转换后的 GROMACS 结构重新生成 `path.pdb`。

问：MetaDynamics FES 不收敛 > (1) 运行更长时间（每个 walker 更多 ns）。(2) 检查山丘是否正在沉积（HILLS 文件应持续增长）。(3) 通过绘制 COLVAR 中的 s(R) 来验证 path CV 是否捕捉了相关的构象变化。

偏差总结

#	参数	SI 值	我们的值	严重程度	备注
1	Lambda (PATHMSD)	0.029 A^-2	0.033910 A^-2	HIGH	不同的 MSD；可能是对齐方法差异
2	Heat7 约束	未指定	10.0 kcal/mol/A^2	MEDIUM	SI 为 7 步列出 6 个值
3	SIGMA (PLUMED)	未报告	0.05	MEDIUM	ADAPTIVE 模式会自动校正
4	Gaussian 版本	09	16c02	LOW	相同的基组和方法
5	AMBER 版本	16	24p3	LOW	相同的 ff14SB 力场
6	GROMACS 版本	5.1.2	2026.0	LOW	相同的 PLUMED 接口

文档生成日期：*2026-04-02* 已验证阶段：*0–5 (UNC Longleaf)* 草稿阶段：*6–9*（待执行）

参考文献